

Universidad Nacional Autónoma de México

Facultad de Ciencias

Título de la Tesis

Tesis presentada para obtener el grado de

Licenciado en Matemáticas

Presentado por:

Hector Mauricio Salazar Ledesma

Director de Tesis:

Nombre del Director

Ciudad Universitaria, México

2 de octubre de 2026

Índice general

1. Introducción	4
2. Gráficas y Superficies	5
2.1. Superficies	5
2.1.1. Espacio topológico	5
2.1.2. Espacio de Hausdorff	7
2.1.3. Compacidad	8
2.1.4. Segundo axioma de numerabilidad	10
2.1.5. Variedades topológicas	11
2.1.6. Complejos simpliciales y triangulaciones	13
2.2. Gráficas	16
2.2.1. Gráficas simples y propiedades básicas	16
2.2.2. Representación de gráficas	18
2.2.3. Subgráficas	19
2.2.4. Caminos y ciclos	20
2.2.5. Árboles	21
3. Redes Neuronales Gráficas	25
3.1. Redes Neuronales	25
3.1.1. Conceptos básicos	25
3.1.2. Entrenamiento de redes neuronales	29
3.1.3. Backpropagation	31
3.2. Redes convolucionales CNN	34
3.2.1. Operación de convolución	34
3.2.2. Propiedades de convolución	36
3.2.3. Filtros y mapas de características	37
3.2.4. Arquitectura de una CNN	39
3.3. Redes Neuronales Gráficas	40
3.3.1. Codificación y predicción	41
3.3.2. Propagación de Mensajes Neuronales	42
3.3.3. La GNN fundamental	44
3.3.4. Arquitectura y simetrías de una GNN	44
3.4. Redes gráficas convolucionales (GCN)	45
4. Construcción de algoritmo de muestreo	47
4.1. Construcción de superficies a partir de símlices	47
4.2. Transformación de superficie a gráfica	49
4.3. Árbol generador mínimo y recorrido auxiliar	50
4.4. Muestreo por esferas	52
4.5. Reasignación de adyacencias	57
4.6. Método general para la reducción de nodos	59

4.7. Análisis de complejidad	61
5. Resultados	63
5.1. Marco experimental	63
5.2. Arquitecturas utilizadas	64
5.2.1. Fourier Neural Operator	64
5.2.2. Dynamic Graph CNN	65
5.2.3. PointNet	66
5.2.4. PointNet++	67
5.2.5. Variantes con reducción estructural	68
5.3. Análisis de resultados	69
5.4. Conclusiones	70
A. Definiciones básicas	72
B. Código implementado en Python	73

Capítulo 1

Introducción

Aquí va a ir la introducción

Capítulo 2

Gráficas y Superficies

La teoría de gráficas y el estudio de superficies constituyen dos áreas fundamentales de las matemáticas que, en principio, podrían parecer independientes, sin embargo se encuentran estrechamente relacionadas cuando una superficie se describe mediante triángulos y complejos simpliciales. Esta relación permite traducir ciertos problemas topológicos al lenguaje combinatorio de las gráficas, lo cual resulta naturalmente útil para el desarrollo de algoritmos orientados al análisis y la clasificación de superficies. La representación computacional de una superficie requiere distinguir el objeto topológico de las estructuras discretas utilizadas para estudiarlo. En esta tesis, una triangulación contiene vértices, aristas y caras; a partir de los vértices y aristas se obtiene una gráfica que permite organizar relaciones de adyacencia. Este paso proporciona una entrada combinatoria de los modelos del Capítulo 3 y del procedimiento de reducción de nodos del Capítulo 4. El presente capítulo establece primero las condiciones que definen una superficie y su representación mediante complejos simpliciales. Después introduce las nociones de vecindario, representación de adyacencias, subgráfica y árbol generador mínimo, necesarias para la explicación de los métodos posteriores. De esta forma, la distinción entre superficie, complejo y gráfica permitirá precisar que información recibe el algoritmo y qué propiedades topológicas no pueden deducirse únicamente de la reducción de nodos. Los fundamentos topológicos se basan en los textos [3, 11]; para el estudio de superficies y complejos simpliciales se consideran [10, 12]. La exposición de la teoría de gráficas se sigue de [4, 8]. A lo largo del capítulo, las definiciones se presentan de manera formal y rigurosa, acompañadas de comentarios explicativos y algunos ejemplos que permitirán interpretar su significado y utilidad dentro del desarrollo del método principal de esta tesis.

2.1. Superficies

Para comprender adecuadamente los objetos de estudio que se revisarán a lo largo de esta tesis, comenzaremos por introducir uno de los conceptos fundamentales que utilizaremos desde el inicio: el de superficie. Para definir que es una superficie necesitaremos primero definir algunas propiedades topológicas relacionadas con el comportamiento local del espacio, así como condiciones de separación y numerabilidad. Asimismo la noción de compacidad permitirá precisar qué se entiende por una superficie cerrada. Las definiciones métricas de abierto, bola y vecindad se recogen en el Apéndice A; en la presente sección extenderemos este lenguaje al contexto más general de espacios topológicos.

2.1.1. Espacio topológico

De manera intuitiva una topología especifica qué subconjuntos se consideran abiertos y permite expresar continuidad y proximidad sin necesidad de recurrir a una medida de

distancia.

Definición 1 (Espacio topológico). Sea X un conjunto. Una **topología** sobre X es una colección $\mathcal{T}$ de subconjuntos de X que satisface los siguientes axiomas:

1. $\emptyset \in \mathcal{T}$ y $X \in \mathcal{T}$.
2. Si $\{U_i\}_{i \in I}$ es una familia arbitraria de elementos de $\mathcal{T}$, entonces

$$\bigcup_{i \in I} U_i \in \mathcal{T}.$$

3. Si $U_1, \dots, U_n \in \mathcal{T}$, entonces

$$\bigcap_{j=1}^n U_j \in \mathcal{T}.$$

A la pareja $(X, \mathcal{T})$ se le denomina **espacio topológico**, y a los elementos de $\mathcal{T}$ se les llama **conjuntos abiertos**.

Si $Y \subseteq X$, la **topología de subespacio** en Y está formada por los conjuntos $Y \cap U$, con U abierto en X . En particular, un abierto de Y no tiene que ser abierto en X . Una **vecindad** de un punto x es un conjunto que contiene un abierto que contiene a x ; esta noción extiende la definición métrica del Apéndice A.

Ejemplo 1. Sea X un conjunto de dos elementos $X = \{x, y\}$, podemos definir una topología sobre este conjunto de la siguiente forma:

Tomemos los conjuntos abiertos: $\mathcal{T} = \{X, \emptyset, \{x\}\}$

Entonces observamos que:

$$\{x\} \cup X = X, \{x\} \cup \emptyset = \{x\}, X \cup \emptyset = X, \{x\} \cap X = \{x\}, \{x\} \cap \emptyset = \emptyset, X \cap \emptyset = \emptyset$$

Por tanto $(X, \mathcal{T})$ es una topología: contiene al vacío y a X , y sus uniones e intersecciones permanecen en la colección. Este espacio se conoce como espacio de Sierpiński y servirá para ilustrar la condición de separación de Hausdorff.

Otro de los conceptos en el que nos auxiliaremos para construir y tratar las superficies trabajadas es el concepto de conjunto cerrado, el cual no es más que la contraparte de la definición de conjunto abierto. Notaremos que la relación entre ambos es directa. Los conjuntos cerrados se definen mediante sus complementos en el espacio considerado.

Definición 2 (Conjunto cerrado). Sea X un espacio topológico. Decimos que un subconjunto A de X es **cerrado** si el conjunto $X - A$ es abierto.

Ejemplo 2. El conjunto $A = \{(x, y) \in \mathbb{R}^2 \mid x \geq 0, y \geq 0\}$ es cerrado en la topología usual de $\mathbb{R}^2$.

Demostración. Observamos que su complemento es

$$\mathbb{R}^2 - A = ((-\infty, 0) \times \mathbb{R}) \cup (\mathbb{R} \times (-\infty, 0)).$$

Ambos semiplanos son abiertos, por lo que su unión es abierta y A es cerrado. La Figura 2.1 muestra ambas regiones. □

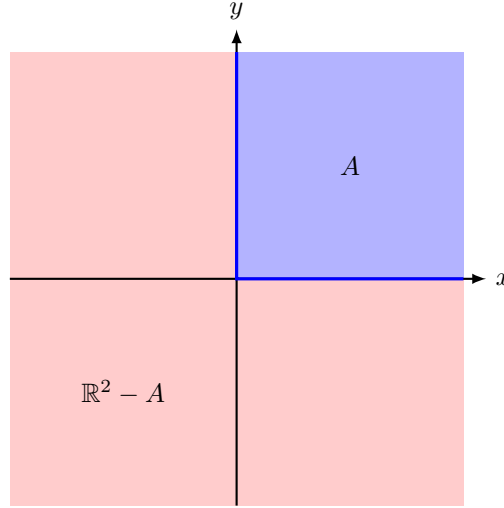


Figura 2.1: El cuadrante cerrado A incluye los semiejes positivos; su complemento es abierto. Se muestra una porción de ambas regiones.

Las siguientes nociones describen un subconjunto respecto de su espacio ambiente. Esta precisión será necesaria para distinguir la frontera de un conjunto del borde de una superficie. Notaremos que a partir de las definiciones de interior, cerradura y frontera podemos llegar de forma natural a la consideración de un axioma para espacios topológicos, el cual abordaremos más adelante.

Definición 3 (Interior de un conjunto). Sea $(X, \mathcal{T})$ un espacio topológico y $A \subseteq X$. El *interior* de A es la unión de todos los abiertos contenidos en A :

$$\text{Int } A = \bigcup \{U \in \mathcal{T} \mid U \subseteq A\}.$$

Proposición 1. Sea $A \subseteq X$. Entonces A es abierto en X si y sólo si $\text{Int } A = A$.

Definición 4 (Cerradura de un conjunto). Sea A un subconjunto de un espacio topológico X . Su *cerradura* es la intersección de todos los cerrados de X que contienen a A :

$$\bar{A} = \bigcap \{C \subseteq X \mid C \text{ es cerrado en } X, A \subseteq C\}.$$

Proposición 2. Sea $A \subseteq X$. Entonces A es cerrado en X si y sólo si $\bar{A} = A$.

Definición 5 (Frontera de un conjunto). Sea A un subconjunto de un espacio topológico X . Un punto $x \in \bar{A} \cap \overline{X - A}$ se denomina *punto frontera* de A ; al conjunto $\bar{A} \cap \overline{X - A}$ se le llama *frontera* de A .

2.1.2. Espacio de Hausdorff

Notemos que, en cierto modo, las definiciones anteriores nos hablan del comportamiento de los puntos respecto a un conjunto. Con la definición 3 podemos describir cuándo un punto "pertenece completamente" a la región; la 4 nos indica cuándo un punto está lo suficientemente "cerca" de ella; y la 5 señala que un punto está justo "en el límite" de dicho conjunto. A partir de estas ideas surge de forma natural la condición que define

un **espacio de Hausdorff**: que dos puntos distintos "no se tocan", es decir, que existen entornos abiertos disjuntos alrededor de cada uno, cuyas cerraduras tampoco se intersecan. En consecuencia, sus interiores quedan totalmente separados. Este requisito cobra especial relevancia al pasar de ejemplos familiares, como $\mathbb{R}$ o $\mathbb{R}^2$, donde cada conjunto puntual $\{x_0\}$ es automáticamente cerrado, a topologías más generales en las que ese hecho ya no se cumple. En el contexto de esta tesis, la definición de superficie exige que dos puntos distintos puedan separarse mediante abiertos disjuntos. Esta condición se expresa con la propiedad de Hausdorff.

Definición 6 (Espacio de Hausdorff). *Un espacio topológico $(X, \mathcal{T})$ es **de Hausdorff** si, para cualesquiera $x_1 \neq x_2$ en X , existen abiertos U_1, U_2 tales que $x_1 \in U_1$, $x_2 \in U_2$ y $U_1 \cap U_2 = \emptyset$.*

Ejemplo 3. *El espacio de Sierpiński no es de Hausdorff: el único abierto que contiene a y es X , que interseca a todo abierto que contiene a x . En cambio, $\mathbb{R}^n$ con su topología usual es de Hausdorff, pues dos puntos distintos admiten bolas abiertas disjuntas de radio menor que la mitad de su distancia. Sus subespacios heredan esta propiedad al intersectar dichas bolas con el subespacio.*

Teorema 1. *Sea X un espacio de Hausdorff. Sea U un conjunto con un número finito de puntos. Entonces U es cerrado en X .*

Demostración. Sean x, x_0 en X tales que $x \neq x_0$. Como X es de Hausdorff entonces existen vecindades U, V tales que $U \cap V = \emptyset$. Como U no interseca al conjunto x_0 entonces el punto x no puede pertenecer a la cerradura del conjunto x_0 . Como consecuencia $\overline{\{x_0\}} = x_0$, por lo tanto es cerrado. $\square$

2.1.3. Compacidad

Otra de las propiedades importantes que necesitamos para entender los objetos centrales de este escrito, es el concepto de compacidad. La compacidad es una condición global: de manera intuitiva podemos decir que un espacio es compacto si podemos "cubrirlo" por completo por conjuntos más pequeños pero que en la suma, den el espacio total. De cualquier cubierta abierta debe poder extraerse una subfamilia finita que siga cubriendo el espacio. Se utilizará al definir las superficies cerradas y al interpretar realizaciones de complejos finitos.

Definición 7 (Cubierta abierta). *Una familia $\mathcal{A} = \{A_i\}_{i \in I}$ de abiertos de un espacio topológico X es una **cubierta abierta** de X si*

$$X = \bigcup_{i \in I} A_i.$$

Definición 8 (Subcubierta finita). *Sea $\mathcal{A}$ una cubierta abierta de X . Una **subcubierta** es una subfamilia $\mathcal{A}' \subseteq \mathcal{A}$ que también cubre a X . Si $\mathcal{A}'$ tiene un número finito de elementos, es una **subcubierta finita**.*

Por último buscamos cubrir en su totalidad de manera controlada y finita por estos objetos decritos, así la definición de espacio compacto queda de la siguiente forma:

Definición 9 (Espacio compacto). *Un espacio topológico X es **compacto** si toda cubierta abierta de X admite una subcubierta finita.*

Ejemplo 4. El subespacio $X = \{0\} \cup \{1/n \mid n \in \mathbb{Z}_+\}$ de $\mathbb{R}$ es compacto.

Demostración. Sea $\mathcal{A}$ una cubierta abierta de X , es decir,

$$X \subseteq \bigcup_{A \in \mathcal{A}} A.$$

Como $0 \in X$ y $\mathcal{A}$ es una cubierta abierta, existe $A_0 \in \mathcal{A}$ tal que $0 \in A_0$. Por lo tanto, existe $\epsilon > 0$ tal que

$$X \cap (-\epsilon, \epsilon) \subseteq A_0.$$

Como la sucesión $\{\frac{1}{n}\}_{n \in \mathbb{N}}$ converge a 0, podemos elegir $N \geq 2$ suficientemente grande de manera que

$$\frac{1}{N} < \epsilon.$$

Entonces, para todo $n \geq N$,

$$0 < \frac{1}{n} \leq \frac{1}{N} < \epsilon,$$

por lo que

$$\frac{1}{n} \in X \cap (-\epsilon, \epsilon) \subseteq A_0.$$

De esta manera, los puntos

$$\frac{1}{N}, \frac{1}{N+1}, \frac{1}{N+2}, \dots$$

junto con 0, quedan cubiertos por A_0 .

De esta forma, solo quedan por cubrir los puntos

$$1, \frac{1}{2}, \frac{1}{3}, \dots, \frac{1}{N-1}.$$

Como $\mathcal{A}$ es una cubierta de X , para cada $k = 1, \dots, N-1$ existe un conjunto $A'_k \in \mathcal{A}$ tal que

$$\frac{1}{k} \in A'_k.$$

Por lo tanto,

$$X \subseteq A_0 \cup A'_1 \cup A'_2 \cup \dots \cup A'_{N-1}.$$

Así hemos obtenido una subcubierta finita de $\mathcal{A}$. En consecuencia, toda cubierta abierta de X admite una subcubierta finita y, por tanto, X es compacto. $\square$

La Figura 2.2 ilustra cómo un entorno de 0 cubre la cola de la sucesión y sólo deja un número finito de puntos por cubrir.

El siguiente resultado permite comprobar la compacidad de un subespacio utilizando abiertos del espacio ambiente.

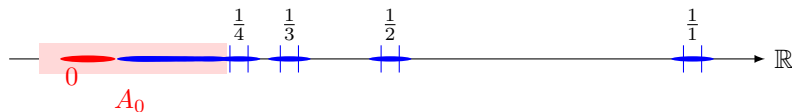


Figura 2.2: Esquema de una subcubierta finita de X . Los intervalos representan abiertos relativos al intersectarlos con X ; sólo se dibujan algunos términos de la sucesión.

Lema 1. *Sea Y un subespacio de un espacio topológico X . Entonces Y es compacto si y sólo si toda cubierta de Y por abiertos de X admite una subfamilia finita que cubre a Y .*

Demostración. Si Y es compacto y $\{A_i\}_{i \in I}$ es una cubierta de Y por abiertos de X , los conjuntos $A_i \cap Y$ forman una cubierta abierta en Y . Una subcubierta finita de ésta proporciona una subfamilia finita de los A_i que cubre a Y .

Recíprocamente, sea $\{U_i\}_{i \in I}$ una cubierta por abiertos de Y . Para cada i existe un abierto A_i de X con $U_i = A_i \cap Y$. Por hipótesis, una subfamilia finita de los A_i cubre a Y ; las intersecciones correspondientes forman una subcubierta finita de los U_i . Por tanto, Y es compacto. $\square$

La relación entre compacidad y separación permitirá distinguir ser un subconjunto cerrado del espacio ambiente y ser una superficie cerrada.

Teorema 2. *Todo subespacio compacto Y de un espacio de Hausdorff X es cerrado en X .*

Demostración. Si $Y = \emptyset$, la afirmación es inmediata. En otro caso, fijemos $x_0 \in X - Y$. Para cada $y \in Y$ existen abiertos disjuntos U_y, V_y que contienen a x_0 y a y , respectivamente. Los V_y cubren a Y ; por el Lema 1, algunos $V_{y_1}, \dots, V_{y_n}$ bastan para cubrirlo. El abierto

$$U = \bigcap_{i=1}^n U_{y_i}$$

contiene a x_0 y es disjunto de $\bigcup_{i=1}^n V_{y_i}$, por lo que $U \subseteq X - Y$. Así, $X - Y$ es abierto y Y es cerrado. $\square$

2.1.4. Segundo axioma de numerabilidad

Por último, definiremos otra de las propiedades que nos sirven como base para llegar a definir nuestro objetivo, la cual es la de **numerabilidad**. Para definir variedades se requiere una base numerable de abiertos para todo el espacio. Esta condición se cumple en los subespacios de $\mathbb{R}^n$ que se emplean para representar las superficies. Esta propiedad nos permite establecer una relación entre los puntos de un espacio topológico y las vecindades que estos poseen. En particular, nos interesa saber si cada punto tiene una base de vecindades numerable, lo que nos permitirá trabajar con espacios más manejables y estructurados.

Definición 10. *(Base de vecindades)*

*Sea X un espacio topológico y $x \in X$. Decimos que una familia $\mathcal{B}_x$ de vecindades de x es una **base de vecindades** de x si para cualquier vecindad U de x existe una vecindad $V \in \mathcal{B}_x$ tal que $V \subset U$*

Definición 11. *(Base numerable)* *Sea un espacio topológico X . Se dice que X tiene una base numerable en x si existe una base de vecindades numerable.*

Definición 12 (Segundo axioma de numerabilidad). Una **base** de un espacio topológico $(X, \mathcal{T})$ es una familia $\mathcal{B} \subseteq \mathcal{T}$ tal que, para todo abierto U y todo $x \in U$, existe $B \in \mathcal{B}$ con $x \in B \subseteq U$. Si existe una base numerable, se dice que X satisface el **segundo axioma de numerabilidad** o que es 2-numerable.

Ejemplo 5. Las bolas de centros en $\mathbb{Q}^n$ y radios racionales positivos forman una base numerable de $\mathbb{R}^n$. Sus intersecciones con un subespacio $Y \subseteq \mathbb{R}^n$ forman una base numerable de Y .

2.1.5. Variedades topológicas

Para definir una superficie de forma rigurosa, las condiciones de Hausdorff y numerabilidad se complementan con una condición local: cada punto debe admitir coordenadas en un modelo euclidiano. Recordemos que una aplicación entre espacios topológicos es **continua** si la preimagen de todo abierto es abierta, y que un **homeomorfismo** es una biyección continua cuya inversa también es continua. Dos espacios homeomorfos tienen la misma estructura topológica.

Primero se introducen las cartas y el borde en términos de espacios topológicos; después se reúnen las condiciones que definen una variedad. Fijemos un entero $n \geq 1$. Además de $\mathbb{R}^n$, utilizaremos el semiespacio

$$\mathbb{H}^n = \{(x_1, \dots, x_n) \in \mathbb{R}^n \mid x_n \geq 0\},$$

con su topología relativa. Así, $V \subseteq \mathbb{H}^n$ es abierto en $\mathbb{H}^n$ si $V = O \cap \mathbb{H}^n$ para algún abierto O de $\mathbb{R}^n$.

Definición 13 (Carta local). Sea X un espacio topológico. Una **carta local de dimensión** n es un par (U, φ) , donde U es un abierto no vacío de X y

$$\varphi : U \longrightarrow V$$

es un homeomorfismo sobre un conjunto V abierto en $\mathbb{R}^n$ o abierto en $\mathbb{H}^n$ con su topología relativa. En el primer caso se denomina **carta euclidiana**; en el segundo, **carta modelada en $\mathbb{H}^n$** . Para $x \in U$, se dice que (U, φ) es una carta alrededor de x , y los componentes de $\varphi(x)$ son sus **coordenadas locales**. Se denota por $\varphi_n(x)$ su última coordenada.

Una carta modelada en $\mathbb{H}^n$ puede tener su imagen contenida en la región $x_n > 0$; por ello, utilizar este modelo no exige la presencia de puntos de borde.

Definición 14 (Borde). Sea X un espacio topológico tal que cada punto pertenece al dominio de una carta modelada en $\mathbb{H}^n$, para una misma dimensión $n \geq 1$. Un punto $x \in X$ es un **punto de borde** si existe una de estas cartas (U, φ) alrededor de x tal que $\varphi_n(x) = 0$. El **borde** de X es el conjunto

$$\partial X = \{x \in X \mid x \text{ es un punto de borde}\}.$$

La condición anterior no depende de la carta elegida. Esta propiedad se conoce como **invariancia topológica del borde** y se obtiene del teorema de invariancia del dominio; para su formulación en variedades, véase [?, Teorema 2.59]. La justificación es local: si una

carta enviara un punto a $x_n = 0$ y otra a $x_n > 0$, el cambio de coordenadas, restringido a una bola alrededor de la segunda imagen contenida en la imagen del solapamiento y en la región $x_n > 0$, sería una aplicación continua e inyectiva desde un abierto de $\mathbb{R}^n$ hacia $\mathbb{R}^n$. La invariancia del dominio obligaría a que su imagen fuera abierta, lo cual es imposible para un conjunto contenido en $\mathbb{H}^n$ que contiene un punto con última coordenada cero.

Con estas nociones, se puede formular la definición de variedad topológica incorporando las condiciones de separación y numerabilidad ya establecidas.

Definición 15 (Variedad topológica). *Sea $n \geq 1$ un entero. Un espacio topológico no vacío X es una **variedad topológica de dimensión n con posible borde** si es de Hausdorff, 2-numerable y cada punto pertenece al dominio de una carta modelada en $\mathbb{H}^n$. Si $\partial X = \emptyset$, se dice que es una **variedad topológica sin borde**.*

Los puntos de $X - \partial X$ admiten cartas euclidianas, obtenidas al restringir las cartas a la región con última coordenada positiva. Recíprocamente, un punto que admite una carta euclidiana no es un punto de borde, por la invariancia del dominio. Así, en el caso sin borde, la condición local equivale a exigir que cada punto tenga un entorno abierto homeomorfo a un abierto de $\mathbb{R}^n$. En adelante, el término **variedad topológica**, sin otro calificativo, se referirá a una variedad sin borde.

Definición 16 (Superficie). *Una variedad topológica de dimensión 2 es una **superficie**; cuando se permite borde se indica explícitamente. Una superficie es **cerrada** si es compacta y no tiene borde.*

El borde de una superficie es una propiedad de su estructura de variedad, mientras que la frontera de la Definición 5 depende del espacio ambiente. Por ejemplo, la esfera $S^2 = \{(x, y, z) \in \mathbb{R}^3 \mid x^2 + y^2 + z^2 = 1\}$ no tiene borde, aunque su frontera en $\mathbb{R}^3$ es toda la esfera. Asimismo, ser cerrado como subconjunto no basta para ser una superficie cerrada: también se requieren la condición local de superficie y la compacidad. El Teorema 1 asegura, por ejemplo, que una nube finita de puntos es cerrada en $\mathbb{R}^3$, sin convertirla en una superficie.

Ejemplo 6. *La gráfica de la función $z = x^2 - y^2$,*

$$S = \{(x, y, z) \in \mathbb{R}^3 \mid z = x^2 - y^2\},$$

es una superficie sin borde, ilustrada en la Figura 2.3.

Demostración. Como subespacio de $\mathbb{R}^3$, S es de Hausdorff y 2-numerable. La proyección

$$\Pi : S \longrightarrow \mathbb{R}^2, \quad \Pi(x, y, z) = (x, y),$$

es continua y biyectiva, y su inversa

$$\Pi^{-1}(x, y) = (x, y, x^2 - y^2)$$

es continua porque sus componentes son polinomios. Por tanto, S es homeomorfa a $\mathbb{R}^2$ y satisface la condición local de la Definición 15. No es una superficie cerrada, pues no es compacta: es un subconjunto no acotado de $\mathbb{R}^3$. $\square$

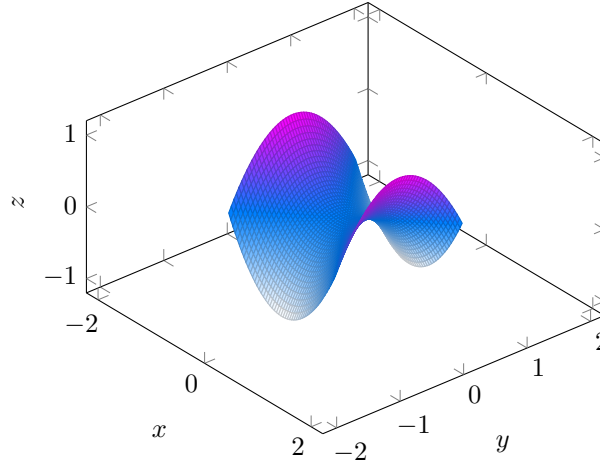


Figura 2.3: Porción de la superficie $z = x^2 - y^2$, conocida como silla de montar. Su dimensión es dos, aunque se represente en $\mathbb{R}^3$.

2.1.6. Complejos simpliciales y triangulaciones

Una vez definida la noción de superficie, interesa describirla mediante estructuras discretas adecuadas para su análisis computacional. Los **complejos simpliciales** permiten organizar los vértices, las aristas y las caras de una triangulación. Como la metodología posterior utiliza mallas finitas, nos restringimos a complejos finitos y distinguimos su estructura combinatoria, su realización geométrica y la gráfica asociada.

Definición 17 (Complejo simplicial). *Sea V un conjunto finito no vacío cuyos elementos llamaremos **vértices**. Un **complejo simplicial** sobre V es una colección $\mathcal{K}$ de subconjuntos no vacíos de V , llamados **símplices**, que satisface las siguientes propiedades:*

1. *Para todo vértice $v \in V$, el conjunto $\{v\}$ pertenece a $\mathcal{K}$.*
2. *Si $\sigma \in \mathcal{K}$ y $\tau \subseteq \sigma$ con $\tau \neq \emptyset$, entonces $\tau \in \mathcal{K}$.*

Notemos que la dimensión de un símplex σ es $|\sigma| - 1$ y la del complejo es la máxima dimensión de sus símplexes. Así, los 0-símplices son vértices, los 1-símplices son aristas y los 2-símplices son caras triangulares. En general, una **cara** de σ es cualquier subconjunto no vacío suyo; en una malla triangular se reserva habitualmente el término cara para los 2-símplices.

De esta forma para pasar del complejo abstracto a un espacio topológico, damos una representación geométrica a cada uno de sus símplexes. Al ser V finito, esta construcción puede realizarse en un espacio euclidiano.

Definición 18 (Símplex geométrico asociado). *Sea $\mathcal{K}$ un complejo simplicial sobre un conjunto finito de vértices V . Denotamos por $\mathbb{R}^V$ el espacio euclidiano de las familias de números reales $t = (t_v)_{v \in V}$. Para cada símplex $\sigma \in \mathcal{K}$, su **símplex geométrico asociado** es*

$$\Delta_\sigma = \left\{ (t_v)_{v \in V} \in \mathbb{R}^V \left| \begin{array}{l} t_v \geq 0 \text{ para todo } v \in V, \\ \sum_{v \in V} t_v = 1, \\ t_v = 0 \text{ si } v \notin \sigma \end{array} \right. \right\}.$$

Los coeficientes t_v , con $v \in \sigma$, se denominan **coordenadas baricéntricas** del punto respecto de los vértices de σ .

Si σ tiene uno, dos o tres vértices, entonces Δ_σ es, respectivamente, un punto, un segmento o un triángulo que incluye su interior y sus lados. Su dimensión es $|\sigma| - 1$. La notación $|\sigma|$ conserva aquí su significado de cardinalidad del símplice abstracto.

Definición 19 (Realización geométrica). *Sea $\mathcal{K}$ un complejo simplicial finito sobre V . Su realización geométrica es el espacio*

$$|\mathcal{K}| = \bigcup_{\sigma \in \mathcal{K}} \Delta_\sigma \subseteq \mathbb{R}^V,$$

dotado de la topología de subespacio de $\mathbb{R}^V$.

Esta construcción representa las relaciones entre las caras del complejo. En efecto, para $\sigma, \tau \in \mathcal{K}$,

$$\Delta_\sigma \cap \Delta_\tau = \begin{cases} \Delta_{\sigma \cap \tau}, & \text{si } \sigma \cap \tau \neq \emptyset, \\ \emptyset, & \text{si } \sigma \cap \tau = \emptyset. \end{cases}$$

Por tanto, las piezas se unen precisamente por las caras comunes prescritas por el complejo abstracto. La Figura 2.4 ilustra realizaciones de complejos de dimensiones uno y dos; en particular, muestra cómo dos triángulos pueden compartir una arista.

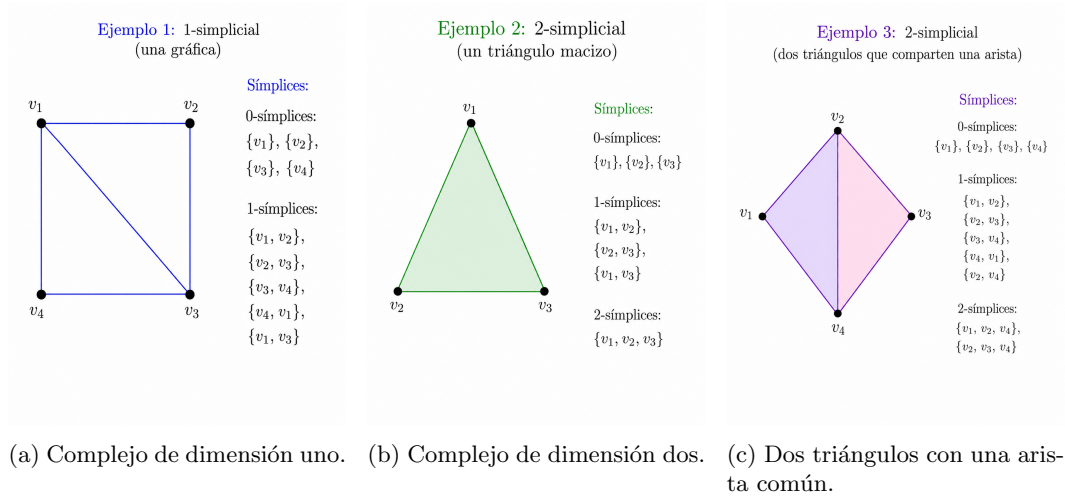


Figura 2.4: Ejemplos de complejos simpliciales.

Definición 20 (Triangulación finita de una superficie). *Sea S una superficie, con o sin borde. Una triangulación finita de S es un par $(\mathcal{K}, h)$, donde $\mathcal{K}$ es un complejo simplicial finito de dimensión dos y*

$$h : |\mathcal{K}| \longrightarrow S$$

es un homeomorfismo.

El homeomorfismo establece la correspondencia topológica entre la superficie y la realización del complejo [10]. Una malla geométrica que aproxima una superficie requiere justificar esta correspondencia; disponer de triángulos por sí solo no la garantiza.

Como $\mathcal{K}$ es finito, $|\mathcal{K}|$ es una unión finita de símlices compactos y, por tanto, es compacto. En consecuencia, una superficie que admite una triangulación finita es necesariamente compacta. Esta condición no implica que la superficie carezca de borde. Si se representa mediante un subespacio homeomorfo de $\mathbb{R}^3$, dicho subespacio es cerrado por el Teorema 2; ser cerrado en el espacio ambiente tampoco determina la ausencia de borde como superficie.

Las coordenadas baricéntricas de la Definición 18 deben distinguirse de las posiciones de los vértices en $\mathbb{R}^3$ empleadas en el Capítulo 4. La realización en $\mathbb{R}^V$ proporciona un modelo canónico del complejo. Para representar el mismo espacio topológico mediante segmentos y triángulos en $\mathbb{R}^3$, se requiere que los símlices no se degeneren y que sus intersecciones sean exactamente las caras comunes indicadas por $\mathcal{K}$; una asignación arbitraria de posiciones no garantiza estas condiciones.

En una triangulación finita $(\mathcal{K}, h)$, la notación computacional $K = (V, E, F)$ organiza los símlices por dimensión: V registra los vértices, mientras que

$$E = \{\sigma \in \mathcal{K} \mid |\sigma| = 2\}, \quad F = \{\sigma \in \mathcal{K} \mid |\sigma| = 3\}$$

registran las aristas y las caras triangulares, respectivamente. Esta terna describe la misma información combinatoria que la familia $\mathcal{K}$ de la Definición 17; el homeomorfismo h especifica su relación con la superficie S .

Definición 21 (Característica de Euler). *Sea $(\mathcal{K}, h)$ una triangulación finita de una superficie S , cuyo complejo se representa por $K = (V, E, F)$. La **característica de Euler** de S es*

$$\chi(S) = |V| - |E| + |F|.$$

Este valor es un invariante topológico: no depende de la triangulación elegida [11].

En el caso de una superficie cerrada, conexa y orientable, se tiene $\chi(S) = 2 - 2g$, donde g es su **género**, interpretado como el número de asas de su modelo topológico: la esfera tiene género cero y el toro género uno [11]. Aquí, conexa significa que no es unión de dos abiertos disjuntos no vacíos; en una triangulación, la orientabilidad puede expresarse eligiendo orientaciones cíclicas de sus triángulos que induzcan sentidos opuestos sobre cada arista compartida. Estas hipótesis son necesarias para utilizar la relación indicada. Euler y género describen las clases de superficies consideradas en EuLearn [5] y en el Capítulo 5.

Para describir las relaciones de adyacencia utilizadas por los algoritmos posteriores, se consideran únicamente los símlices de dimensión a lo más uno.

Definición 22 (1-esqueleto). *Sea $\mathcal{K}$ un complejo simplicial. Su **1-esqueleto** es el complejo simplicial*

$$\mathcal{K}^{(1)} = \{\sigma \in \mathcal{K} \mid |\sigma| \leq 2\},$$

formado por todos sus vértices y aristas.

La pareja $G = (V, E)$ registra estos vértices y aristas y da lugar a la gráfica que se estudia en la siguiente sección. A diferencia de $\mathcal{K}$, esta representación no registra el conjunto F de caras triangulares.

2.2. Gráficas

A partir de las definiciones previas, es posible introducir nuevos objetos que servirán como base para la caracterización formal de las superficies de interés. De este modo, comenzaremos definiendo las **gráficas**, que son estructuras matemáticas compuestas por vértices y aristas. El **1-esqueleto** de un complejo está formado por sus vértices y aristas. En la notación anterior se representa mediante $G = (V, E)$ y proporciona las adyacencias utilizadas en los capítulos siguientes. Un 1-símplice corresponde a una arista y un 2-símplice a un triángulo con sus caras; al pasar a G , el conjunto F deja de registrarse explícitamente. Por ello, una transformación de la gráfica no determina por sí sola una transformación de la triangulación ni la conservación de su característica de Euler.

Las siguientes secciones formalizan esta representación y fija las convenciones para estudiar vecindarios, recorridos y árboles.

2.2.1. Gráficas simples y propiedades básicas

Definición 23 (Gráfica simple). *Una **gráfica simple** es un par $G = (V, E)$, donde V es el conjunto de vértices y*

$$E \subseteq E(V) := \{\{u, v\} \mid u, v \in V, u \neq v\}$$

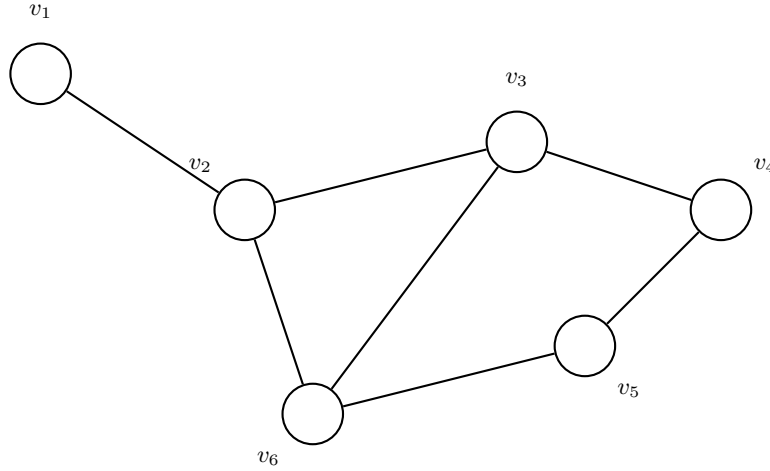
es el conjunto de aristas. En esta tesis se consideran gráficas finitas y no dirigidas.

Se escribirá también $V(G)$ y $E(G)$ para indicar la gráfica a la que pertenecen los conjuntos. El par no ordenado $\{u, v\}$ se abrevia como uv . La definición excluye bucles y aristas múltiples, pero no exige que todos los pares de vértices distintos sean aristas. En el contexto computacional, los vértices se denominan también nodos.

Derivado de la definición anterior, podemos establecer algunas propiedades de las gráficas que permitirán caracterizar y analizarlas. Estas propiedades incluyen el orden y el tamaño de la gráfica, así como las relaciones entre los vértices y aristas.

Definición 24 (Orden y tamaño). *Para una gráfica G , tenemos que:*

1. $\nu_G = |V(G)|$ es llamado el orden de G .
2. $\varepsilon_G = |E(G)|$ es llamado el tamaño de G .
3. Para cada arista $e = uv \in E$ los vértices u y v son llamados extremos de la arista.
4. Los vértices u y v son adyacentes o vecinos si $e = uv \in E$.
5. Dos aristas distintas son adyacentes si tienen un extremo común.

Figura 2.5: Ejemplo de gráfica simple G .

En la Figura 2.5, $\nu_G = 6$ y $\varepsilon_G = 7$. Los extremos de v_1v_2 son v_1 y v_2 , que son vecinos; las aristas v_1v_2 y v_2v_6 son adyacentes porque comparten el vértice v_2 .

Una de las propiedades locales de una gráfica que será de especial utilidad a lo largo de este trabajo es la noción de vecindario. Esta permite describir el entorno inmediato de cada vértice a partir de los vértices adyacentes a él y, con ello, identificar las relaciones locales de conectividad presentes en la gráfica.

Definición 25 (Vecindario). Sea $v \in V(G)$ un vértice de la gráfica G . El **vecindario** de v es el conjunto dado por:

$$N_G(v) = \{u \in V(G) \mid vu \in E(G)\}$$

Dada una gráfica $G = (V, E)$, podemos notar que el vecindario $N_G(v)$ depende de las aristas, no de una distancia entre coordenadas. Esta distinción será relevante tanto para la propagación de mensajes del Capítulo 3 como para la selección espacial de candidatos del Capítulo 4. La Figura 2.6 muestra los vecinos de v_2 .

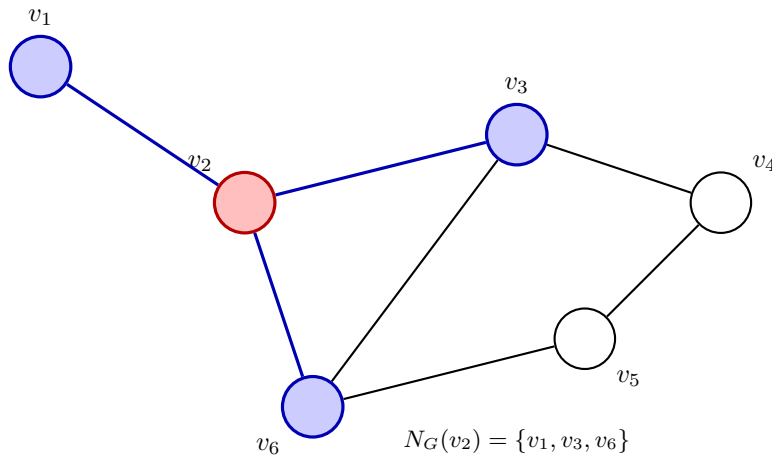


Figura 2.6: Vecindario del vértice v_2 en la gráfica G . El vecindario resultante es $N_G(v_2) = \{v_1, v_3, v_6\}$

Otra de las definiciones que sirven tanto para caracterizar así como dar importancia a los nodos de una gráfica es el **grado de un vértice**. Esta propiedad cuantifica la conectividad local de cada vértice dentro de la gráfica y la utilizaremos previamente en la normalización de adyacencias de las redes gráficas convolucionales del Capítulo 3.

Definición 26 (Grado de un vértice). *Para $v \in V(G)$, el **grado** de v es el número de sus vecinos:*

$$d_G(v) = |N_G(v)|.$$

De la definición anterior podemos notar que para cualquier vértice v , podría suceder que v no tenga vecinos o por el otro lado que el resto de los vértices de G sean sus vecinos, recordando que v no puede ser vecino de sí mismo. Así, se tiene que:

$$0 \leq d_G(v) \leq n - 1$$

donde n es el orden la gráfica G .

Definición 27 (Vértice aislado). *Un vértice v es **aislado** si $d_G(v) = 0$.*

Aislar un vértice elimina sus conexiones, pero conserva el vértice en $V(G)$. Esta operación se distinguirá de la eliminación de nodos en el Capítulo 4.

2.2.2. Representación de gráficas

Existen diversas formas de representar gráficas, podemos encontrarlas en su forma visual como la de la figura 2.5, o bien podemos encontrarlas en su forma matricial conocida como **matriz de adyacencia**. En esta matriz, las filas y columnas representan los vértices de la gráfica, y los elementos de la matriz indican si existe una arista entre los vértices correspondientes. A continuación, definiremos formalmente la matriz de adyacencia de una gráfica simple y presentaremos un ejemplo para ilustrar su uso.

Definición 28 (Matriz de adyacencia). *Sea G una gráfica simple con vértices ordenados $V(G) = \{v_1, \dots, v_{\nu_G}\}$. Su **matriz de adyacencia** es $A(G) = [a_{ij}] \in \{0, 1\}^{\nu_G \times \nu_G}$, donde*

$$a_{ij} = \begin{cases} 1, & \text{si } v_i v_j \in E(G), \\ 0, & \text{en otro caso.} \end{cases}$$

Ejemplo 7. *La matriz de adyacencia de la figura 2.5 quedaría de la siguiente forma:*

$$A(G) = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \end{pmatrix} \quad (2.1)$$

Notamos que la matriz de adyacencia es una matriz simétrica, ya que si v_i y v_j son vecinos entonces v_j y v_i también lo son, por lo tanto $a_{ij} = a_{ji}$. Además, notamos que la diagonal

principal de la matriz de adyacencia es cero, ya que no existe una arista que conecte un vértice consigo mismo en una gráfica simple.

La matriz determina qué vértices intercambian información en los modelos del Capítulo 3. Si cada vértice tiene coordenadas en $\mathbb{R}^3$, éstas pueden registrarse como características iniciales en una matriz distinta, con una fila por vértice. Las coordenadas y las adyacencias cumplen funciones diferentes: las primeras describen posiciones y las segundas conexiones.

Una alternativa que mejora notablemente la optimización de memoria es la representación de gráficas mediante listas de adyacencia. Este modelo asocia a cada vértice una colección de sus nodos colindantes, lo que permite un acceso más rápido a la información de la conexiones entre los vértices al prescindir de una matriz completa.

Definición 29 (Lista de adyacencia). Sea $G = (V, E)$ una gráfica simple. Su **representación por listas de adyacencia** es la familia $\{L_v\}_{v \in V}$, donde L_v enumera, sin repeticiones, los elementos de $N_G(v)$. El orden de cada lista es una elección de la representación, no una propiedad de la gráfica.

Ejemplo 8. Sea $G = (V, E)$ la gráfica de la figura 2.5, entonces la lista de adyacencia de G sería:

Vértice	Lista de adyacencia
v_1	(v_2)
v_2	(v_1, v_3, v_6)
v_3	(v_2, v_4, v_6)
v_4	(v_3, v_5)
v_5	(v_4, v_6)
v_6	(v_2, v_3, v_5)

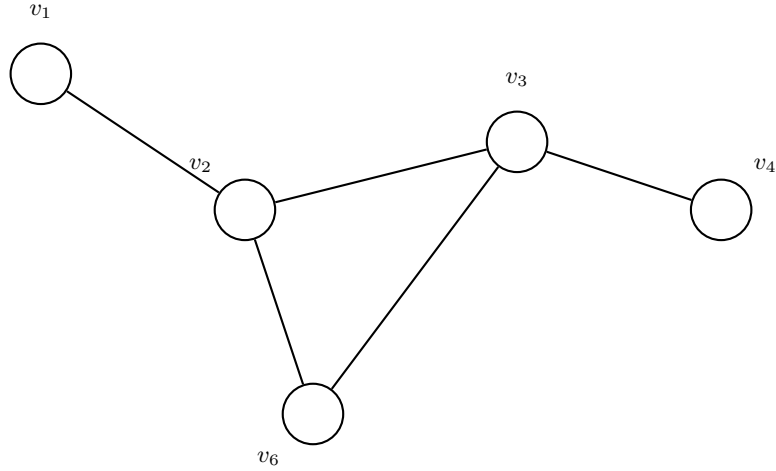
Una matriz completa almacena ν_G^2 entradas, mientras que las listas almacenan ν_G registros de vértices y $2\varepsilon_G$ entradas de vecinos. Consultar una entrada de la matriz permite comprobar una adyacencia directamente; recorrer todos los vecinos exige revisar una fila o, en la otra representación, recorrer su lista. Estas diferencias se retomarán al analizar la implementación del Capítulo 4.

2.2.3. Subgráficas

Debido a los métodos que se utilizarán más adelante en el estudio de los objetos y debido al gran número de datos a procesar, es necesario introducir conceptos que se pueden emplear para el estudio local de las gráficas. Para distinguir las operaciones sobre una gráfica es necesario precisar qué significa conservar parte de sus vértices y aristas. Estas nociones permiten separar la construcción de un árbol auxiliar de la reducción posterior que puede reasignar conexiones.

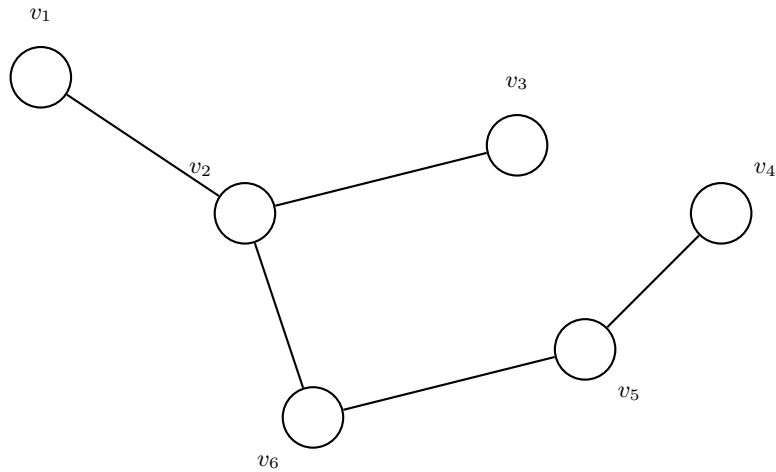
Definición 30 (Subgráfica). Sean $G = (V, E)$ y $H = (V_H, E_H)$ gráficas simples. Se dice que H es una **subgráfica** de G , y se escribe $H \subseteq G$, si $V_H \subseteq V$ y $E_H \subseteq E$.

La Figura 2.7 muestra una subgráfica de la gráfica G de la Figura 2.5.

Figura 2.7: Ejemplo de subgráfica de G .

Definición 31 (Subgráfica generadora). *Una subgráfica $H \subseteq G$ es **generadora** si $V(H) = V(G)$.*

Una subgráfica generadora conserva todos los vértices, aunque puede omitir aristas, como en la Figura 2.8. Por tanto, obtenerla no reduce el orden de la gráfica.

Figura 2.8: Subgráfica generadora H de la gráfica G de la Figura 2.5.

Definición 32 (Subgráfica inducida). *Sea G una gráfica simple y $V_H \subseteq V(G)$. La **subgráfica inducida** por V_H es la gráfica H con conjunto de vértices V_H y*

$$E(H) = \{uv \in E(G) \mid u, v \in V_H\}.$$

Así, H conserva todas las aristas de G entre los vértices elegidos. La Figura 2.9 muestra la subgráfica inducida por $\{v_2, v_3, v_6\}$. Si un procedimiento añade una arista que no pertenece a $E(G)$, su salida no es una subgráfica de G ; esta distinción se utilizará al describir la reasignación de adyacencias.

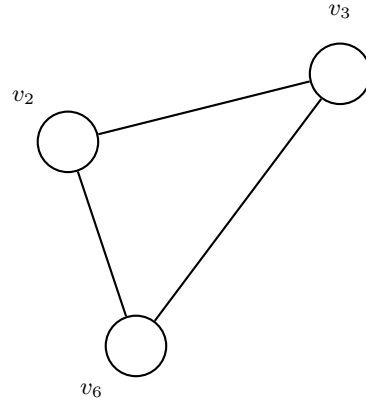


Figura 2.9: Ejemplo de una gráfica inducida.

2.2.4. Caminos y ciclos

Uno de los conceptos fundamentales que se introducirán y además se implementarán en la creación de los algoritmos de 4 es el de recorridos, caminos y ciclos. Los recorridos permiten describir cómo se visitan los vértices de una gráfica. La distinción entre recorrido, camino y ciclo servirá para formalizar la conexidad y demostrar las propiedades de los árboles utilizados en el muestreo.

Definición 33 (Recorrido, camino y ciclo). Sea G una gráfica simple. Un **recorrido** es una secuencia $W = (u_0, u_1, \dots, u_k)$, con $k \geq 0$ entero y $u_i \in V(G)$ para $i = 0, \dots, k$, tal que $u_{i-1}u_i \in E(G)$ para $i = 1, \dots, k$. Su **longitud** es $l(W) = k$; se permiten repeticiones de vértices y aristas.

1. El recorrido es **cerrado** si $u_0 = u_k$.
2. Es un **camino** si todos sus vértices son distintos.
3. Es un **ciclo** si $k \geq 3$, $u_0 = u_k$ y $u_0, \dots, u_{k-1}$ son distintos.

Un camino o un ciclo también puede identificarse con la subgráfica formada por sus vértices y aristas. En estos casos su longitud coincide con el número de aristas de dicha subgráfica.

Las Figuras 2.10 y 2.11 ilustran un camino de longitud cuatro y un ciclo de longitud cuatro, respectivamente.

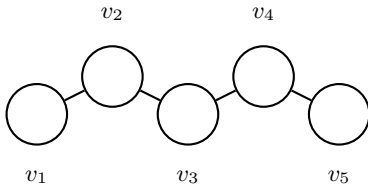


Figura 2.10: Ejemplo de un camino.

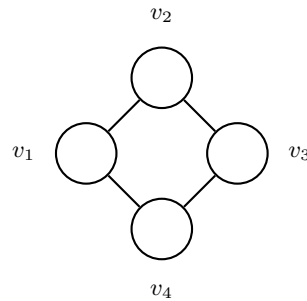


Figura 2.11: Ejemplo de un ciclo.

2.2.5. Árboles

Los árboles permiten organizar recorridos sin ciclos. En el Capítulo 4 se empleará un árbol generador como estructura auxiliar que conserva todos los vértices de la gráfica original; la reducción de nodos ocurrirá después. Para justificar su construcción se necesitan la conexidad y la relación entre puentes y ciclos.

Definición 34 (Gráfica conexa). *Una gráfica $G = (V, E)$ es **conexa** si $V \neq \emptyset$ y para todo par $u, v \in V$ existe un camino de u a v . Para $u = v$ se admite el camino de longitud cero.*

La Figura 2.12 presenta una gráfica en la que todos los pares de vértices están conectados por caminos.

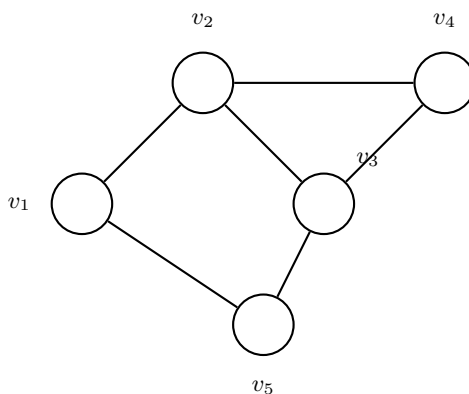


Figura 2.12: Ejemplo de una gráfica conexa.

Una **componente conexa** es una subgráfica conexa maximal por inclusión: no está contenida propiamente en otra subgráfica conexa de G . Las componentes particionan el conjunto de vértices, y G es conexa si y sólo si tiene una componente.

Definición 35 (Punto). *Sea $e \in E(G)$. Se escribe $G - e = (V(G), E(G) - \{e\})$. La arista e es un **puente** si su eliminación aumenta el número de componentes conexas. En particular, si G es conexa, e es un puente si y sólo si $G - e$ es desconexa.*

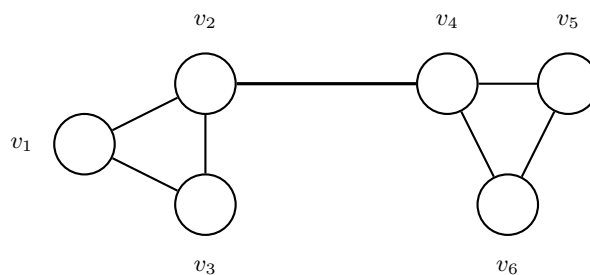


Figura 2.13: Ejemplo de una gráfica con un puente (arista de corte).

En la Figura 2.13, la arista v_2v_4 es un puente: al eliminarla, los vértices $\{v_1, v_2, v_3\}$ y $\{v_4, v_5, v_6\}$ quedan en componentes distintas.

Proposición 3. Una arista $e \in E(G)$ es un puente si y sólo si no pertenece a ningún ciclo de G .

Demostración. Sea $e = uv$. Si $G - e$ contiene un camino de u a v , cualquier recorrido que utilizaba e puede sustituirla por ese camino; de un recorrido entre dos vértices se obtiene un camino suprimiendo los tramos cerrados que repiten vértices. Por tanto, no se separa ninguna componente. Si no existe tal camino, u y v , antes conectados por e , quedan en componentes distintas. Así, e es un puente exactamente cuando sus extremos no están conectados en $G - e$.

Si e pertenece a un ciclo, las restantes aristas del ciclo forman un camino de u a v en $G - e$, por lo que e no es un puente. Recíprocamente, si e no es un puente, existe un camino de u a v en $G - e$; junto con e forma un ciclo que contiene a e . $\square$

Definición 36 (Gráfica acíclica). Una gráfica es **acíclica** si no contiene ciclos. También se denomina *bosque*.

Definición 37 (Árbol). Un árbol es una gráfica acíclica y conexa. La Figura 2.14 muestra un ejemplo.

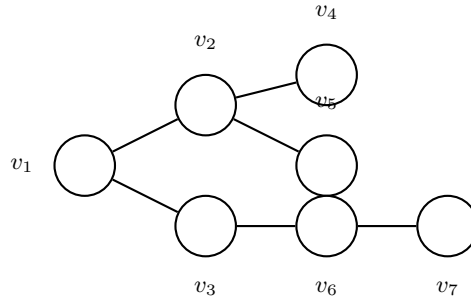


Figura 2.14: Ejemplo de una gráfica acíclica (un árbol).

De la Proposición 3 y la Definición 37 se obtiene el siguiente corolario.

Corolario 1. Una gráfica conexa es un árbol si y sólo si todas sus aristas son puentes.

Las siguientes caracterizaciones justifican que un árbol conecte todos sus vértices utilizando exactamente una arista menos que su orden.

Teorema 3. Sea T una gráfica finita con $\nu_T \geq 1$. Los siguientes enunciados son equivalentes:

1. T es un árbol.
2. Cualquiera dos vértices de T están conectados por un único camino.
3. T es acíclica y $\varepsilon_T = \nu_T - 1$.

Demostración. (1) $\Rightarrow$ (2). La conexidad garantiza la existencia de un camino. Si hubiera dos caminos distintos entre los mismos extremos, al seguirlos desde un punto donde se separan hasta su primer reencuentro se obtendría un ciclo. Esto contradice que T sea acíclica.

(2) $\Rightarrow$ (3). Un ciclo proporcionaría dos caminos distintos entre dos de sus vértices, de modo que T es acíclica. Para contar sus aristas procedemos por inducción sobre $n = \nu_T$. Si $n = 1$, no hay aristas y la igualdad se cumple. Si $n \geq 2$, consideremos un camino de longitud máxima y uno de sus extremos v . Un vecino de v fuera del camino permitiría prolongarlo, y un vecino distinto del vértice consecutivo dentro del camino produciría un ciclo. Por tanto, $d_T(v) = 1$.

Al eliminar v y su única arista incidente se obtiene la subgráfica inducida $T - v$. El camino único entre dos vértices restantes no puede pasar por v como vértice interior, pues su grado es uno. Así, $T - v$ conserva la propiedad de camino único y, por la hipótesis de inducción,

$$\varepsilon_{T-v} = \nu_{T-v} - 1 = n - 2.$$

Al restituir la arista eliminada resulta $\varepsilon_T = n - 1$.

(3) $\Rightarrow$ (1). Sean $T_1, \dots, T_k$ las componentes conexas de T . Cada una es conexa y acíclica, por lo que es un árbol. Por las implicaciones ya demostradas, $\varepsilon_{T_i} = \nu_{T_i} - 1$. En consecuencia,

$$\nu_T - 1 = \varepsilon_T = \sum_{i=1}^k (\nu_{T_i} - 1) = \nu_T - k.$$

Luego $k = 1$ y T es conexa; como también es acíclica, es un árbol. $\square$

Definición 38 (Árbol generador). *Dada una gráfica conexa G , un **árbol generador** de G es una subgráfica $T \subseteq G$ que es generadora y es un árbol. En particular, $V(T) = V(G)$.*

Teorema 4. *Toda gráfica finita conexa contiene un árbol generador.*

Demostración. Partimos de $H_0 = G$. Mientras H_i tenga una arista e_i que no sea puente, definimos $H_{i+1} = H_i - e_i$. Cada paso conserva todos los vértices y la conexidad. Como el número de aristas es finito y disminuye en cada paso, el procedimiento termina en una subgráfica generadora conexa H cuyas aristas son todas puentes. Por el Corolario 1, H es un árbol. La condición de puente se comprueba en cada gráfica actual; no se eliminan simultáneamente todas las aristas que no eran puentes en G . $\square$

Para elegir entre los posibles árboles generadores se pueden asignar pesos a las aristas y minimizar su suma.

Definición 39 (Árbol generador mínimo). *Sea G una gráfica finita conexa con una función de pesos $w : E(G) \rightarrow \mathbb{R}$. El peso de un árbol generador T es*

$$w(T) = \sum_{e \in E(T)} w(e).$$

*Un **árbol generador mínimo** es un árbol generador T^* tal que*

$$w(T^*) = \min\{w(T) \mid T \text{ es un árbol generador de } G\}.$$

La familia de árboles generadores es finita y no vacía, de modo que este mínimo existe, aunque puede no ser único. Si todas las aristas tienen peso uno, cada árbol generador pesa $\nu_G - 1$ por el Teorema 3; en ese caso todos son mínimos. Esta observación será relevante para la aplicación de Kruskal a una matriz de adyacencia binaria en el Capítulo 4.

El capítulo ha distinguido la superficie de su triangulación y de la gráfica que registra el 1-esqueleto. Las caras permiten calcular la característica de Euler de la triangulación; las coordenadas y adyacencias proporcionan datos para los modelos del Capítulo 3. A su vez, los árboles generadores sustentan el recorrido auxiliar del Capítulo 4, sin eliminar vértices. Estas conexiones establecen el lenguaje de los métodos posteriores; la reducción computacional y sus efectos en la clasificación requieren su propio análisis y no implican por sí mismos conservación topológica.

Capítulo 3

Redes Neuronales Gráficas

El Capítulo 2 estableció las representaciones combinatorias de las superficies mediante complejos simpliciales y gráficas. El presente capítulo introduce los modelos neuronales que permiten transformar los atributos de esas representaciones en predicciones. Se estudian primero las capas y su entrenamiento, después las operaciones locales de las redes convolucionales y, finalmente, la agregación de información sobre vecindarios de una gráfica.

Esta organización proporciona los fundamentos para distinguir dos etapas posteriores: el muestreo del Capítulo 4, que modifica la representación de entrada, y la clasificación del Capítulo 5, que utiliza representaciones aprendidas. En particular, será necesario diferenciar las coordenadas de los nodos de sus encajes, y la equivarianza de las capas de la invarianza de una predicción global. La Figura 3.1 muestra la composición de capas que sirve como punto de partida.

Los fundamentos de redes neuronales y convoluciones se presentan siguiendo [6]; el tratamiento de las redes neuronales gráficas se apoya en [7].

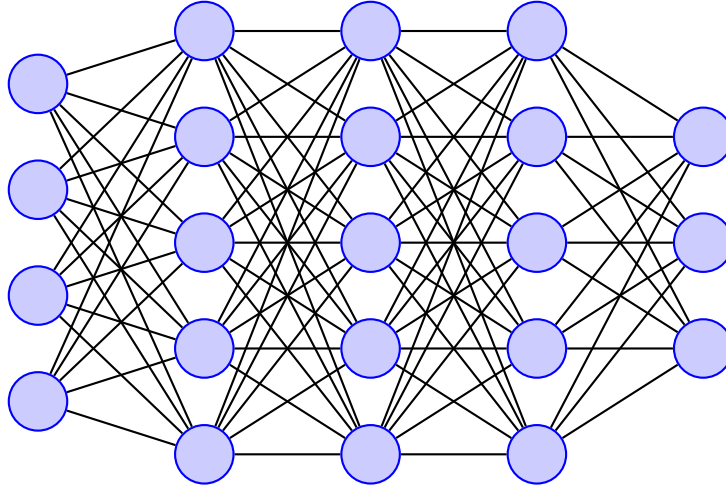


Figura 3.1: Ejemplo de red neuronal con múltiples capas

3.1. Redes Neuronales

3.1.1. Conceptos básicos

Una red neuronal se construye mediante la composición de transformaciones parametrizadas y funciones de activación. Estas operaciones aparecen tanto en los clasificadores como en las funciones que generan y actualizan mensajes sobre gráficas. Se comienza con el perceptrón como modelo de decisión binaria y después se generaliza su transformación afín a capas con activaciones adecuadas para el entrenamiento.

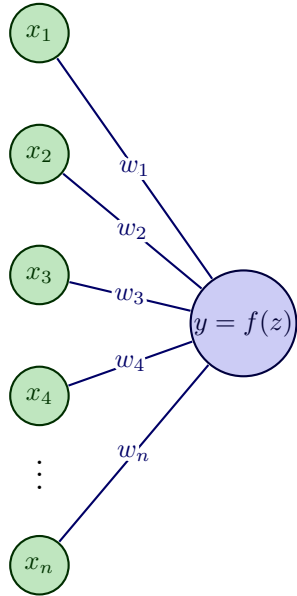
Definición 40 (Perceptrón). Sea $\mathbf{w} \in \mathbb{R}^d$ un vector de pesos, $\mathbf{x} \in \mathbb{R}^d$ un vector de entrada y $b \in \mathbb{R}$ un sesgo (umbral). Definimos la función

$$f : \mathbb{R}^d \rightarrow \{0, 1\}, \quad f(\mathbf{x}) = \chi_{\{\mathbf{y} \in \mathbb{R}^d : \mathbf{w}^T \mathbf{y} + b > 0\}}(\mathbf{x}),$$

donde χ_A denota la función indicadora del conjunto $A \subseteq \mathbb{R}^d$; este uso no corresponde a la característica de Euler de una superficie.

En forma explícita,

$$f(\mathbf{x}) = \begin{cases} 1, & \text{si } \mathbf{w}^T \mathbf{x} + b > 0, \\ 0, & \text{si } \mathbf{w}^T \mathbf{x} + b \leq 0. \end{cases}$$



$$z = \mathbf{w}^T \mathbf{x} + b = \sum_{i=1}^n x_i w_i + b,$$

$$y = f(z) = \begin{cases} 1, & \text{si } z > 0, \\ 0, & \text{si } z \leq 0. \end{cases}$$

Figura 3.2: Ejemplo de perceptrón con suma ponderada, sesgo y activación escalón.

El perceptrón de la Definición 40 separa las entradas mediante el hiperplano $\mathbf{w}^T \mathbf{x} + b = 0$, cuando $\mathbf{w} \neq 0$. Las Figuras 3.2 y 3.3 muestran su operación y su frontera de decisión. La activación escalón da una salida binaria; las capas siguientes utilizan otras activaciones para construir modelos entrenables mediante gradientes.

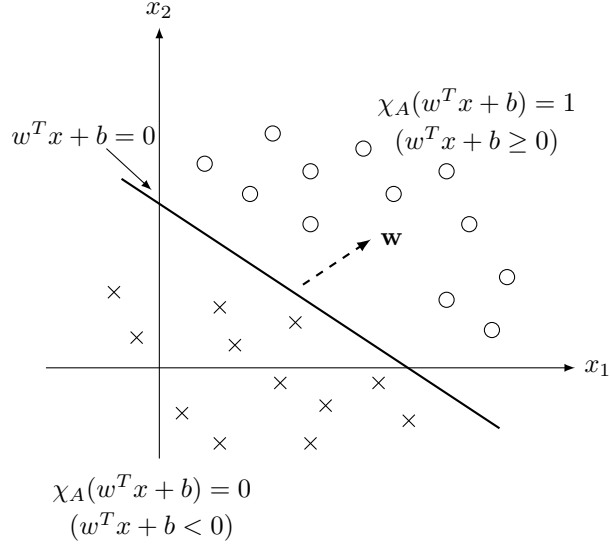


Figura 3.3: Ejemplo de clasificador lineal. Los círculos representan la clase positiva y las cruces la clase negativa. La línea representa la frontera de decisión definida por el perceptrón.

La capa densa generaliza la combinación afín del perceptrón a varias unidades. La no linealidad se aplica después de esta combinación, como se ilustra en la Figura 3.4.

Definición 41 (Capa densa de una red neuronal). *Sea $L \in \mathbb{N}$ el índice de una capa en una red neuronal. Sean: $\mathbf{W}^{(L)} \in \mathbb{R}^{n_L \times n_{L-1}}$ la matriz de pesos que conecta la capa $(L-1)$ con la capa L , $\mathbf{b}^{(L)} \in \mathbb{R}^{n_L}$ el vector de sesgos, $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ una función de activación (aplicada componente a componente).*

Definimos la **capa L -ésima** como la aplicación

$$f^{(L)} : \mathbb{R}^{n_{L-1}} \longrightarrow \mathbb{R}^{n_L},$$

dada por

$$f^{(L)}(\mathbf{x}) = \sigma(\mathbf{W}^{(L)}\mathbf{x} + \mathbf{b}^{(L)}),$$

donde la función de activación σ se aplica de manera componente a componente sobre el vector $\mathbf{W}^{(L)}\mathbf{x} + \mathbf{b}^{(L)}$.

En particular:

$$f^{(0)}(\mathbf{x}) = \mathbf{x}, \quad \mathbf{x} \in \mathbb{R}^{n_0}.$$

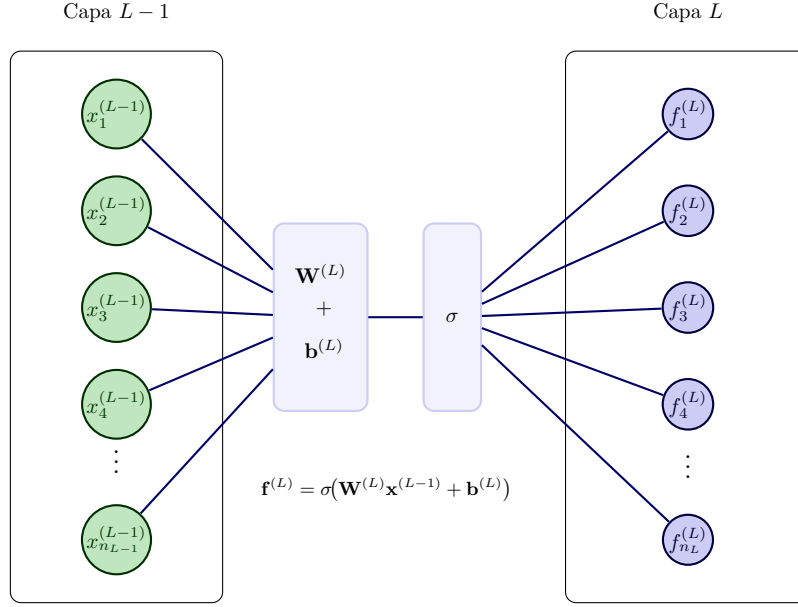


Figura 3.4: Capa de una red neuronal con múltiples neuronas en la capa L .

Observación 1. Una función de activación no lineal σ introduce no linealidad en la red neuronal, lo cual es de importancia para que una red pueda aprender representaciones complejas y no lineales de los datos. Algunas funciones de activación comunes son:

- **Función sigmoide:** $\sigma(x) = \frac{1}{1+e^{-x}}$
- **Función ReLU (Rectified Linear Unit):** $\sigma(x) = \max(0, x)$
- **Función tanh:** $\sigma(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

La composición de las capas de la Definición 41 determina la red completa. En este contexto, una red formada por capas densas se denomina también perceptrón multicapa (MLP).

Definición 42 (Red neuronal). Sea $L \in \mathbb{N}$ el número de capas. Una **red neuronal** con L capas es una función

$$f(\cdot; \theta) : \mathbb{R}^{n_0} \longrightarrow \mathbb{R}^{n_L}$$

definida como la composición

$$f(\mathbf{x}; \theta) = (f^{(L)} \circ f^{(L-1)} \circ \dots \circ f^{(1)})(\mathbf{x}),$$

donde: $\mathbf{x} \in \mathbb{R}^{n_0}$ es el vector de entrada, para cada $i \in \{1, \dots, L\}$, $f^{(i)} : \mathbb{R}^{n_{i-1}} \rightarrow \mathbb{R}^{n_i}$ es la transformación asociada a la i -ésima capa, $\theta = (\mathbf{W}^{(i)}, \mathbf{b}^{(i)})_{i=1}^L$ es la colección de parámetros de la red (matrices de pesos y vectores de sesgo), $\hat{\mathbf{y}} = f(\mathbf{x}; \theta) \in \mathbb{R}^{n_L}$ es la salida de la red.

La capa $f^{(L)}$ es la capa de salida y las capas $f^{(1)}, \dots, f^{(L-1)}$ son ocultas. La entrada es $\mathbf{x}$, no una capa con parámetros entrenables. La Figura 3.5 ilustra el caso de una capa oculta.

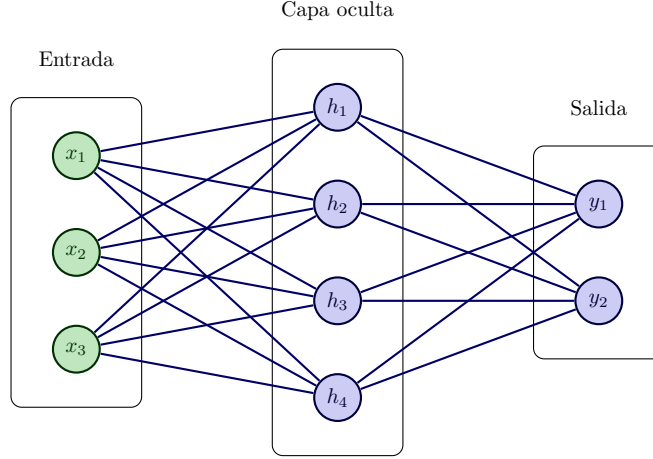


Figura 3.5: Ejemplo de red neuronal con una capa oculta.

3.1.2. Entrenamiento de redes neuronales

Entrenar una red consiste en ajustar sus parámetros a partir de observaciones etiquetadas. Se distinguirán la pérdida de una predicción, su promedio en la distribución de datos y el promedio calculable sobre el conjunto de entrenamiento. Esta distinción permite interpretar la evaluación de clasificación del Capítulo 5 sin identificar el error de entrenamiento con el comportamiento en datos no utilizados para ajustar el modelo.

Definición 43 (Conjunto de entrenamiento). *Sea $\mathcal{X}$ el espacio de entrada (o espacio de características) y sea $\mathcal{Y}$ el espacio de salida (o espacio de etiquetas).*

*Un **conjunto de datos de entrenamiento** es una colección finita de N observaciones empíricas, denotada por*

$$\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\} = \{(x_i, y_i)\}_{i=1}^N,$$

donde cada par $(x_i, y_i) \in \mathcal{X} \times \mathcal{Y}$ representa un ejemplo individual. En este contexto, x_i corresponde a las características observadas del i -ésimo ejemplo y y_i es su valor verdadero o etiqueta asociada.

En la tarea de clasificación de superficies, cada x_i representa una superficie mediante los datos que recibe el modelo, y y_i es la etiqueta de esa observación. El muestreo del Capítulo 4 modifica la representación de x_i ; reducir sus nodos no equivale a retirar observaciones de $\mathcal{D}$.

Definición 44 (Función de pérdida). *Sea $\mathcal{Y}$ el espacio de valores verdaderos y sea $\hat{\mathcal{Y}}$ el espacio de predicciones de un modelo.*

*Una **función de pérdida** es una función*

$$\ell : \mathcal{Y} \times \hat{\mathcal{Y}} \longrightarrow \mathbb{R}_{\geq 0},$$

donde $\ell(y, \hat{y})$ cuantifica la discrepancia entre el valor verdadero $y \in \mathcal{Y}$ y una predicción $\hat{y} \in \hat{\mathcal{Y}}$.

En particular, si

$$f(\cdot; \theta) : \mathcal{X} \longrightarrow \hat{\mathcal{Y}}$$

es una red neuronal parametrizada por $\theta \in \Theta$, donde Θ es el espacio de parámetros, entonces la pérdida asociada a un dato de entrenamiento (x_i, y_i) está dada por

$$\ell(y_i, f(x_i; \theta)).$$

La pérdida puntual se promedia respecto de una distribución de probabilidad para formular el objetivo poblacional del aprendizaje. Se denota esta distribución por $\mathbb{P}$, reservando $\mathcal{D}$ para la colección finita de observaciones.

Definición 45 (Función de riesgo). Sea $\mathbb{P}$ una distribución de probabilidad sobre $\mathcal{X} \times \mathcal{Y}$. Para una pérdida ℓ y un predictor $f(\cdot; \theta)$, se define el **riesgo esperado** por

$$\mathcal{R}(\theta) = \mathbb{E}_{(x,y) \sim \mathbb{P}} [\ell(y, f(x; \theta))],$$

siempre que la pérdida compuesta sea medible e integrable. Bajo esta condición, $\mathcal{R} : \Theta \rightarrow \mathbb{R}_{\geq 0}$ cuantifica la pérdida promedio sobre la distribución de datos.

Como la distribución $\mathbb{P}$ es desconocida, se emplea un promedio sobre las observaciones disponibles.

Definición 46 (Función de riesgo empírico). Dado el conjunto de entrenamiento $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, con $N \geq 1$, el **riesgo empírico** es

$$\hat{\mathcal{R}}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(y_i, f(x_i; \theta)).$$

El problema de minimización del riesgo empírico consiste en buscar parámetros que reduzcan $\hat{\mathcal{R}}$ sobre Θ .

La pérdida ℓ corresponde a una observación, mientras que $\mathcal{R}$ y $\hat{\mathcal{R}}$ son promedios distintos. Reducir la pérdida de entrenamiento no garantiza obtener buena clasificación en datos nuevos. Para ajustar los parámetros se utilizan métodos de optimización que, cuando existen las derivadas necesarias, se apoyan en el gradiente.

Definición 47 (Gradiente). Sea $f : \Omega \rightarrow \mathbb{R}$, con $\Omega \subseteq \mathbb{R}^n$ abierto, diferenciable en $x_0 \in \Omega$. Su **gradiente** en x_0 es el vector columna

$$\nabla f(x_0) = \left(\frac{\partial f}{\partial x_1}(x_0), \dots, \frac{\partial f}{\partial x_n}(x_0) \right)^T.$$

El producto $\nabla f(x_0)^T d$ da la derivada direccional en la dirección d . Si el gradiente es distinto de cero, su opuesto es una dirección de descenso local.

Definición 48 (Descenso por gradiente). Sea $J : \Theta \rightarrow \mathbb{R}$ una función diferenciable, denominada función objetivo, donde $\Theta = \mathbb{R}^n$ representa el espacio de parámetros sin restricciones. Dado un valor inicial $\theta_0 \in \Theta$ y una tasa de aprendizaje $\eta > 0$, el método de descenso por gradiente genera una sucesión $\{\theta_k\}_{k \in \mathbb{N}}$ definida recursivamente por

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} J(\theta_k),$$

donde $\nabla_{\theta} J(\theta_k)$ denota el gradiente de J evaluado en θ_k .

Notamos que el descenso por gradiente es un método iterativo que busca minimizar una función objetivo $J(\theta)$ ajustando los parámetros θ en la dirección opuesta al gradiente de la función. La tasa de aprendizaje η controla el tamaño de los pasos que se dan en cada iteración, lo que afecta el comportamiento del método. La actualización no garantiza por sí sola convergencia ni descenso para cualquier valor de η .

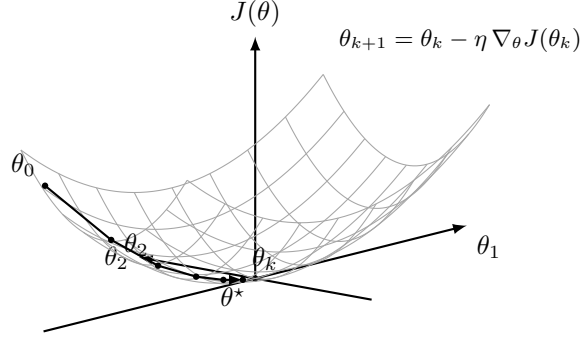


Figura 3.6: Ilustración geométrica del método de descenso por gradiente aplicado a una función objetivo $J : \Theta \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$.

De esta forma podemos describir formalmente el algoritmo de actualización de parámetros por medio del descenso de gradiente.

Algorithm 1 Actualización por descenso del gradiente

Require: Objetivo diferenciable $J : \mathbb{R}^n \rightarrow \mathbb{R}$, $T \geq 1$, $\eta > 0$, $\theta^{(0)}$

```

1: for  $t = 0, \dots, T - 1$  do
2:    $g \leftarrow \nabla J(\theta^{(t)})$ 
3:   if  $g = 0$  then
4:     return  $\theta^{(t)}$ 
5:   end if
6:    $\theta^{(t+1)} \leftarrow \theta^{(t)} - \eta g$ 
7: end for
8: return  $\theta^{(T)}$ 

```

En el entrenamiento puede tomarse $J = \hat{\mathcal{R}}$. El Algoritmo 1 presupone que se conoce su gradiente; la retropropagación permite calcularlo a través de las capas sin ser, por sí misma, una regla de actualización de parámetros.

3.1.3. Backpropagation

La **retropropagación** (*backpropagation*) aplica la regla de la cadena para calcular las derivadas de la pérdida respecto de los parámetros. Se separan el cálculo de activaciones, la propagación de derivadas y la actualización mediante un optimizador. La Figura 3.7 ilustra estos recorridos para una red con una capa oculta.

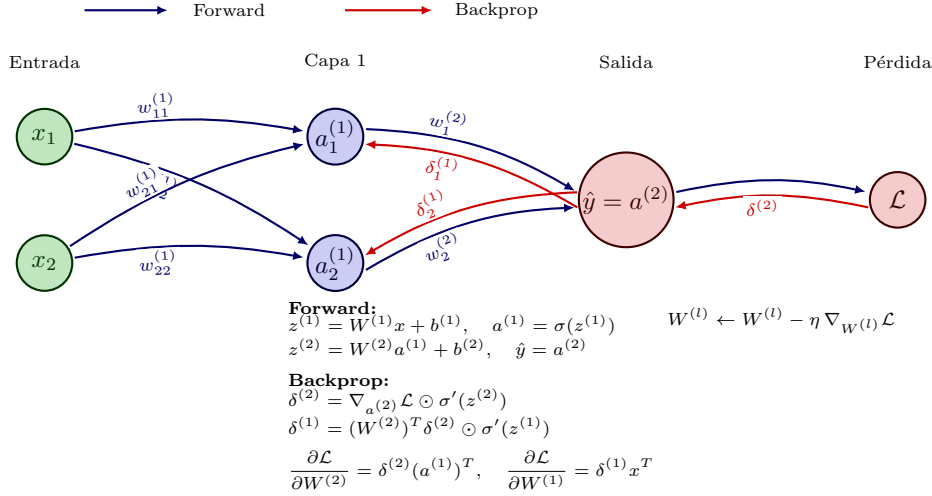


Figura 3.7: Propagación de activaciones hacia la salida (azul) y de derivadas hacia las capas anteriores (rojo). La actualización utiliza los gradientes calculados.

Definición 49 (Gradiente de la función de riesgo empírico). *Se agrupan las entradas de las matrices de pesos y los sesgos en un vector $\theta \in \mathbb{R}^p$. Si las pérdidas compuestas son diferenciables respecto de este vector, entonces*

$$\nabla_{\theta} \widehat{\mathcal{R}}(\theta) = \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \ell(y_i, f(x_i; \theta)).$$

Si, además, la red es diferenciable respecto de sus parámetros y la pérdida respecto de la predicción, la regla de la cadena da

$$\nabla_{\theta} \ell(y_i, f(x_i; \theta)) = (D_{\theta} f(x_i; \theta))^T \nabla_{\hat{y}} \ell(y_i, \hat{y})|_{\hat{y}=f(x_i; \theta)},$$

donde $D_{\theta} f \in \mathbb{R}^{n_L \times p}$ es el jacobiano respecto de todos los parámetros de la red.

La Definición 49 permite calcular el gradiente por observación y promediarlo. Para obtenerlo sin formar el jacobiano completo, se recorre la composición de capas en sentido inverso. A continuación se supone que cada activación se aplica componente a componente y es diferenciable en las preactivaciones consideradas.

Propagación hacia delante. Para una observación $(\mathbf{x}, y)$ se fija $\mathbf{h}^{(0)} = \mathbf{x}$. Para cada capa $l = 1, \dots, L$, se calculan la preactivación y la activación:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{h}^{(l)} = \sigma(\mathbf{z}^{(l)}).$$

La pérdida puntual es $\ell(y, \mathbf{h}^{(L)})$.

Propagación hacia atrás. Se denota por $\delta^{(l)}$ el gradiente de esta pérdida respecto de $\mathbf{z}^{(l)}$. Con $\odot$ para el producto componente a componente, se obtiene

$$\delta^{(L)} = \nabla_{\mathbf{h}^{(L)}} \ell(y, \mathbf{h}^{(L)}) \odot \sigma'(\mathbf{z}^{(L)}),$$

y, para $l = L - 1, \dots, 1$,

$$\delta^{(l)} = (\mathbf{W}^{(l+1)})^T \delta^{(l+1)} \odot \sigma'(\mathbf{z}^{(l)}).$$

Los gradientes de los parámetros de *todas* las capas, incluida la de salida, son

$$\nabla_{\mathbf{W}^{(l)}} \ell = \delta^{(l)} (\mathbf{h}^{(l-1)})^T, \quad \nabla_{\mathbf{b}^{(l)}} \ell = \delta^{(l)}, \quad l = 1, \dots, L.$$

El primer producto es exterior y tiene las dimensiones de $\mathbf{W}^{(l)}$. Todos los gradientes se calculan antes de modificar los parámetros. Si se utiliza ReLU, no existe derivada en cero; el procedimiento requiere una convención en ese punto, por ejemplo asignar el valor cero. Para activaciones vectoriales no separables debe utilizarse su jacobiano, en lugar del producto componente a componente anterior.

El Algoritmo 2 combina estas operaciones con una actualización por observación. Esta variante se denomina descenso estocástico del gradiente cuando las observaciones se seleccionan aleatoriamente. Promediar los gradientes de todas las observaciones recupera el gradiente del riesgo empírico; promediar sólo un lote da la actualización por minilotes.

Algorithm 2 Retropropagación y actualización por observación**Require:** Datos $\mathcal{D}$, capas L , épocas T , tasa $\eta > 0$ y parámetros iniciales

```

1: for cada época  $t = 1, \dots, T$  do
2:   for cada observación  $(\mathbf{x}, y)$  en el orden elegido para la época do
3:      $\mathbf{h}^{(0)} \leftarrow \mathbf{x}$ 
4:     for  $l = 1, \dots, L$  do
5:        $\mathbf{z}^{(l)} \leftarrow \mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}$ 
6:        $\mathbf{h}^{(l)} \leftarrow \sigma(\mathbf{z}^{(l)})$ 
7:     end for
8:     Calcular  $\ell(y, \mathbf{h}^{(L)})$ 
9:      $\delta^{(L)} \leftarrow \nabla_{\mathbf{h}^{(L)}} \ell(y, \mathbf{h}^{(L)}) \odot \sigma'(\mathbf{z}^{(L)})$ 
10:    for  $l = L - 1, \dots, 1$  do
11:       $\delta^{(l)} \leftarrow (\mathbf{W}^{(l+1)})^T \delta^{(l+1)} \odot \sigma'(\mathbf{z}^{(l)})$ 
12:    end for
13:    for  $l = 1, \dots, L$  do
14:       $g_W^{(l)} \leftarrow \delta^{(l)} (\mathbf{h}^{(l-1)})^T$ ;  $g_b^{(l)} \leftarrow \delta^{(l)}$ 
15:    end for
16:    for  $l = 1, \dots, L$  do
17:       $\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta g_W^{(l)}$ 
18:       $\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta g_b^{(l)}$ 
19:    end for
20:  end for
21: end for

```

Estos mecanismos de entrenamiento se aplican también a las transformaciones aprendibles de las arquitecturas posteriores. Lo que cambia entre modelos es, entre otros aspectos, la forma de compartir parámetros y combinar los datos. La convolución permite introducir estas ideas sobre una cuadrícula antes de considerar vecindarios en gráficas.

3.2. Redes convolucionales CNN

Una **red neuronal convolucional (CNN)** procesa datos en cuadrículas mediante operaciones locales con parámetros compartidos. Se estudian como antecedente de la agregación de información y de las simetrías de los modelos sobre gráficas, siguiendo [6]. La exposición distingue la convolución matemática de la operación habitualmente implementada en las capas y conserva las convenciones necesarias para interpretar los esquemas.

3.2.1. Operación de convolución

Para entender las redes convolucionales, es esencial comprender primero la operación matemática de la convolución. A continuación, se presentan las definiciones clave relacionadas con la convolución en diferentes contextos.

Definición 50 (Convolución). Sean $x, w \in L^1(\mathbb{R})$, es decir, funciones medibles absoluta-

mente integrables. Su **convolución** se define por

$$(x * w)(t) = \int_{\mathbb{R}} x(a)w(t-a) da$$

en los puntos donde la integral converge absolutamente. Para estas funciones la expresión está definida casi en todas partes y determina una función de $L^1(\mathbb{R})$.

En el contexto de las redes neuronales convolucionales, el primer término (la función x) se refiere a la entrada, y el segundo término (la función w) es el kernel o filtro.

Dado que en muchos escenarios, el tipo de datos es discreto, se maneja otro tipo de convolución, llamada convolución discreta.

Definición 51 (Convolución discreta). Sean $x, w : \mathbb{Z} \rightarrow \mathbb{R}$ dos sucesiones absolutamente sumables. La **convolución discreta** de x y w , denotada por $x * w$, es la sucesión

$$(x * w) : \mathbb{Z} \rightarrow \mathbb{R}$$

definida, para todo $n \in \mathbb{Z}$, por

$$(x * w)(n) = \sum_{a=-\infty}^{\infty} x(a)w(n-a),$$

siempre que dicha serie sea convergente.

Finalmente, para el uso particular del procesamiento de imágenes en dos dimensiones, podemos utilizar la convolución discreta en dos dimensiones.

Definición 52 (Convolución discreta en dos dimensiones). Sean $\mathbf{I}, \mathbf{K} : \mathbb{Z}^2 \rightarrow \mathbb{R}$ dos funciones (o matrices discretas) absolutamente sumables. La **convolución discreta bidimensional** de $\mathbf{I}$ y $\mathbf{K}$, denotada por $\mathbf{I} * \mathbf{K}$, es la función

$$(\mathbf{I} * \mathbf{K}) : \mathbb{Z}^2 \rightarrow \mathbb{R}$$

definida, para todo $(i, j) \in \mathbb{Z}^2$, por

$$(\mathbf{I} * \mathbf{K})(i, j) = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} \mathbf{I}(m, n) \mathbf{K}(i-m, j-n),$$

siempre que la doble suma sea convergente.

Observación 2. La operación de convolución es conmutativa, es decir,

$$\mathbf{I} * \mathbf{K} = \mathbf{K} * \mathbf{I}.$$

Demostración. La sumabilidad absoluta permite efectuar el cambio de índices $u = i - m$, $v = j - n$. Para cada $(i, j) \in \mathbb{Z}^2$,

$$(\mathbf{I} * \mathbf{K})(i, j) = \sum_{u, v \in \mathbb{Z}} \mathbf{I}(i-u, j-v) \mathbf{K}(u, v) = (\mathbf{K} * \mathbf{I})(i, j).$$

□

La Figura 3.8 ilustra el filtrado local mediante un kernel simétrico respecto de su centro. El cálculo usa ventanas válidas y, por esa simetría, coincide con la correlación cruzada que se distinguirá de la convolución al definir los filtros.

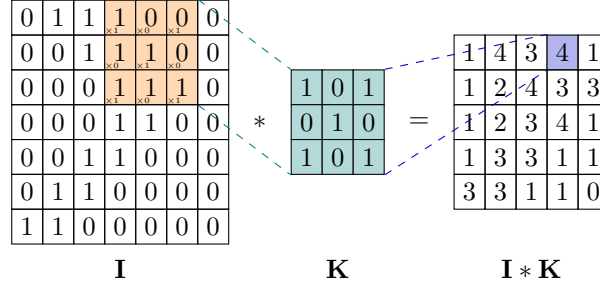


Figura 3.8: Ilustración de la operación de convolución **I** y un kernel **K**.

3.2.2. Propiedades de convolución

Una de las propiedades más importantes de la operación de convolución es su comportamiento bajo ciertas transformaciones, específicamente la invarianza y la equivarianza. Estas propiedades son fundamentales para entender cómo las redes convolucionales pueden aprender características relevantes de los datos de entrada. A continuación, se presentan las definiciones formales de invarianza y equivarianza.

Definición 53 (Invarianza). Sea $f : \mathbf{X} \rightarrow \mathbf{Y}$ una función, $\mathbf{T}$ una transformación en $\mathbf{X}$. Se dice que una función f es invariante bajo $\mathbf{T}$ si:

$$f(\mathbf{T}(x)) = f(x), \quad \forall x \in \mathbf{X}$$

Definición 54 (Equivarianza). Sean $f : \mathbf{X} \rightarrow \mathbf{Y}$ y transformaciones $\mathbf{T}_{\mathbf{X}} : \mathbf{X} \rightarrow \mathbf{X}$ y $\mathbf{T}_{\mathbf{Y}} : \mathbf{Y} \rightarrow \mathbf{Y}$. Se dice que f es **equivariante** respecto de estas transformaciones si

$$f(\mathbf{T}_{\mathbf{X}}(x)) = \mathbf{T}_{\mathbf{Y}}(f(x)), \quad x \in \mathbf{X}.$$

De lo anterior podemos observar que la invarianza implica que la salida de la función no cambia cuando se aplica una transformación a la entrada, mientras que la equivarianza implica que la transformación de la entrada se refleja de manera consistente en la salida. A continuación, demostraremos que la operación de convolución es equivariante bajo traslaciones.

Teorema 5 (Equivarianza de la convolución bajo traslaciones). Sea $f, k \in L^1(\mathbb{R})$ y sea $\mathbf{T}_a$ el operador de traslación definido por

$$(\mathbf{T}_a f)(x) = f(x - a), \quad a \in \mathbb{R}.$$

Entonces, para casi todo $x \in \mathbb{R}$, se cumple que

$$(\mathbf{T}_a f * k)(x) = (f * k)(x - a).$$

Demostración. Por definición de la convolución en $\mathbb{R}$, tenemos que

$$(\mathbf{T}_a f * k)(x) = \int_{\mathbb{R}} (\mathbf{T}_a f)(t) k(x - t) dt.$$

Sustituyendo la definición de la traslación, se obtiene

$$(\mathbf{T}_a f * k)(x) = \int_{\mathbb{R}} f(t - a) k(x - t) dt.$$

Haciendo el cambio de variable $u = t - a$, de modo que $du = dt$ y $t = u + a$, se sigue que

$$(\mathbf{T}_a f * k)(x) = \int_{\mathbb{R}} f(u) k(x - (u + a)) du.$$

Reordenando los términos en el argumento de k , obtenemos

$$(\mathbf{T}_a f * k)(x) = \int_{\mathbb{R}} f(u) k((x - a) - u) du.$$

Por la definición de la convolución, esto equivale a

$$(f * k)(x - a) = \int_{\mathbb{R}} f(u) k((x - a) - u) du.$$

Por lo tanto,

$$(\mathbf{T}_a f * k)(x) = (f * k)(x - a), \quad \text{para casi todo } x \in \mathbb{R}.$$

□

El Teorema 5 expresa equivarianza: trasladar la entrada traslada el resultado de la convolución. No implica que toda CNN sea invariante o equivariante ante cualquier traslación, pues el tratamiento de bordes, el paso del filtro y las capas posteriores pueden modificar esta propiedad. La misma distinción entre una salida que se transforma y una salida que permanece igual se retomará para las permutaciones de nodos.

3.2.3. Filtros y mapas de características

En el contexto de las redes neuronales convolucionales, los **filtros** (o kernels) son matrices de pesos que se utilizan para extraer características locales de los datos de entrada mediante la operación de convolución. A continuación, se presentan las definiciones clave relacionadas con los filtros y los mapas de características.

Definición 55 (Kernel o filtro). *Para una entrada $\mathbf{X} \in \mathbb{R}^{m \times n}$, un **kernel** es una matriz de pesos $\mathbf{K} \in \mathbb{R}^{h \times w}$, con $1 \leq h \leq m$ y $1 \leq w \leq n$. La operación local sin invertir el filtro es la **correlación cruzada**, que se denota aquí por $\star$:*

$$(\mathbf{X} \star \mathbf{K})(i, j) = \sum_{u=1}^h \sum_{v=1}^w \mathbf{X}(i + u - 1, j + v - 1) \mathbf{K}(u, v),$$

para $1 \leq i \leq m - h + 1$ y $1 \leq j \leq n - w + 1$.

El desplazamiento de índices corresponde a una enumeración desde uno y sitúa la esquina

superior izquierda de la ventana en (i, j) . Muchas capas llamadas convolucionales utilizan esta correlación [6]; se reserva $*$ para la convolución matemática. Los filtros aprendidos no tienen que ser simétricos ni estar normalizados para sumar uno. Sus $h \times w$ dimensiones espaciales determinan hw coeficientes por canal.

Definición 56 (Mapa de características). Para $\mathbf{I} \in \mathbb{R}^{m \times n}$ y un filtro $\mathbf{K} \in \mathbb{R}^{h \times w}$, con ventana válida y paso uno, la respuesta lineal es $\mathbf{Z} = \mathbf{I} \star \mathbf{K}$. Al añadir un sesgo $b \in \mathbb{R}$ y aplicar una activación, se obtiene el **mapa de características**

$$\mathbf{F}(i, j) = \sigma(\mathbf{Z}(i, j) + b), \quad \mathbf{F} \in \mathbb{R}^{(m-h+1) \times (n-w+1)}.$$

Si se omiten sesgo y activación, el mapa coincide con la respuesta lineal del filtro.

El paso del filtro y el tratamiento del borde determinan qué ventanas se evalúan y las dimensiones de salida.

Definición 57 (Stride). El **stride** o paso es un entero $s \geq 1$. Para una entrada y un filtro de dimensiones $m \times n$ y $h \times w$, respectivamente, con $h \leq m$ y $w \leq n$, la respuesta con paso s es

$$(\mathbf{I} \star_s \mathbf{K})(i, j) = \sum_{u=1}^h \sum_{v=1}^w \mathbf{I}((i-1)s + u, (j-1)s + v) \mathbf{K}(u, v),$$

con $1 \leq i \leq \lfloor (m-h)/s \rfloor + 1$ y $1 \leq j \leq \lfloor (n-w)/s \rfloor + 1$.

Definición 58 (Padding). El **padding** de amplitud $\rho \in \mathbb{Z}_{\geq 0}$ añade un borde a la entrada. Para relleno con ceros, la aplicación $\mathbf{p}_\rho : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^{(m+2\rho) \times (n+2\rho)}$ se define por

$$\mathbf{p}_\rho(\mathbf{I})(i, j) = \begin{cases} \mathbf{I}(i - \rho, j - \rho), & \rho < i \leq m + \rho, \rho < j \leq n + \rho, \\ 0, & \text{en las demás posiciones del arreglo.} \end{cases}$$

El **pooling** agrega valores en una ventana y puede reducir la resolución espacial de un mapa. Las operaciones de máximo y promedio también se utilizarán sobre familias de características en gráficas y nubes de puntos; allí la región de agregación no tiene que ser una ventana rectangular.

Definición 59 (Max Pooling). Sea $\mathbf{X} \in \mathbb{R}^{m \times n}$ una matriz de entrada, y sean $h, w \in \mathbb{N}$ las dimensiones de la ventana pooling, y $s \in \mathbb{N}$ el parámetro de stride. Definimos la operación de **max pooling** como una función

$$\mathbf{M} : \mathbb{R}^{m \times n} \longrightarrow \mathbb{R}^{p \times q},$$

dada por

$$\mathbf{M}(\mathbf{X})(i, j) = \max_{0 \leq u < h, 0 \leq v < w} \mathbf{X}(1 + (i-1)s + u, 1 + (j-1)s + v),$$

para todo $i = 1, \dots, p$ y $j = 1, \dots, q$, suponiendo $1 \leq h \leq m$, $1 \leq w \leq n$ y $s \geq 1$, donde $p = \lfloor \frac{m-h}{s} \rfloor + 1$ y $q = \lfloor \frac{n-w}{s} \rfloor + 1$.

Definición 60 (Average Pooling). Sea $\mathbf{X} \in \mathbb{R}^{m \times n}$ una matriz de entrada, y sean $h, w \in \mathbb{N}$ las dimensiones de la ventana de pooling, y $s \in \mathbb{N}$ el parámetro de stride. Definimos la

operación de **average pooling** como una función

$$\mathbf{A} : \mathbb{R}^{m \times n} \longrightarrow \mathbb{R}^{p \times q},$$

dada por

$$\mathbf{A}(\mathbf{X})(i, j) = \frac{1}{h \cdot w} \sum_{u=0}^{h-1} \sum_{v=0}^{w-1} \mathbf{X}(1 + (i-1)s + u, 1 + (j-1)s + v),$$

para todo $i = 1, \dots, p$ y $j = 1, \dots, q$, suponiendo $1 \leq h \leq m$, $1 \leq w \leq n$ y $s \geq 1$, donde $p = \lfloor \frac{m-h}{s} \rfloor + 1$ y $q = \lfloor \frac{n-w}{s} \rfloor + 1$.

Con lo anterior hemos definido los conceptos fundamentales relacionados con las redes neuronales convolucionales, incluyendo la operación de convolución, los filtros, los mapas de características, el stride, el padding y las operaciones de pooling. A continuación, exploraremos la arquitectura típica de una CNN y cómo estos componentes se integran para formar una red completa.

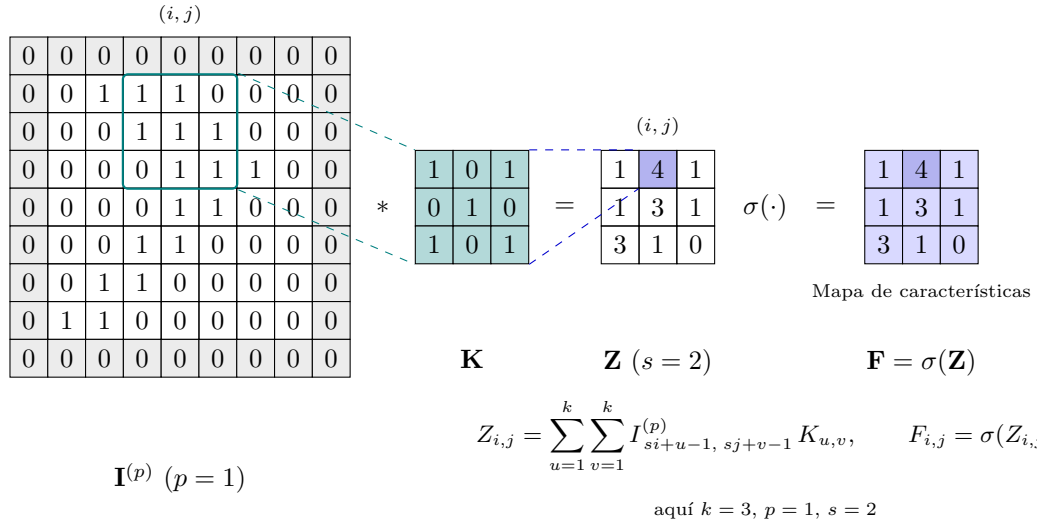


Figura 3.9: Filtrado con padding $p = 1$ y stride $s = 2$. Una entrada de 7×7 y un filtro de 3×3 producen una salida de 4×4 . En este ejemplo el sesgo es cero y σ es ReLU; como $\mathbf{Z}$ tiene entradas no negativas, $\mathbf{F} = \mathbf{Z}$.

3.2.4. Arquitectura de una CNN

A continuación, presentamos la definición formal de la arquitectura de una red neuronal convolucional (CNN).

Definición 61 (Arquitectura básica de una CNN). *Una arquitectura convolucional básica para clasificación se expresa como*

$$f = \mathbf{g} \circ \text{vec} \circ \mathbf{P} \circ \mathbf{C},$$

donde $\mathbf{C}$ produce mapas de características, $\mathbf{P}$ agrega valores mediante pooling, vec aplana el tensor resultante y $\mathbf{g}$ es un clasificador sobre vectores.

Para una entrada $\mathbf{X} \in \mathbb{R}^{m \times n \times d}$, con d canales, un bloque sin padding y con paso uno puede escribirse como

$$\mathbf{C}(\mathbf{X})(i, j, r) = \sigma \left(\sum_{c=1}^d \sum_{u=1}^h \sum_{v=1}^w \mathbf{X}(i+u-1, j+v-1, c) \mathbf{K}(u, v, c, r) + b_r \right).$$

Aquí r identifica un filtro de salida. El pooling se aplica a cada canal según las Definiciones 59 o 60. Después del aplanamiento, $\mathbf{g}$ combina capas densas y una función de salida adecuada a la clasificación. El pooling puede omitirse tomando $\mathbf{P}$ como la identidad. Pueden apilarse varios bloques convolucionales; esta composición no pretende describir todas las arquitecturas CNN.

En clasificación con k clases, una opción para convertir las puntuaciones $a \in \mathbb{R}^k$ en probabilidades es la función vectorial

$$\text{softmax}(a)_j = \frac{e^{a_j}}{\sum_{r=1}^k e^{a_r}}, \quad j = 1, \dots, k.$$

A diferencia de las activaciones escalares, cada componente depende de todas las puntuaciones. La Figura 3.10 muestra un ejemplo de extracción convolucional y clasificación. La localidad y el uso de parámetros compartidos motivan el paso a gráficas, donde el vecindario reemplaza a la ventana rectangular.

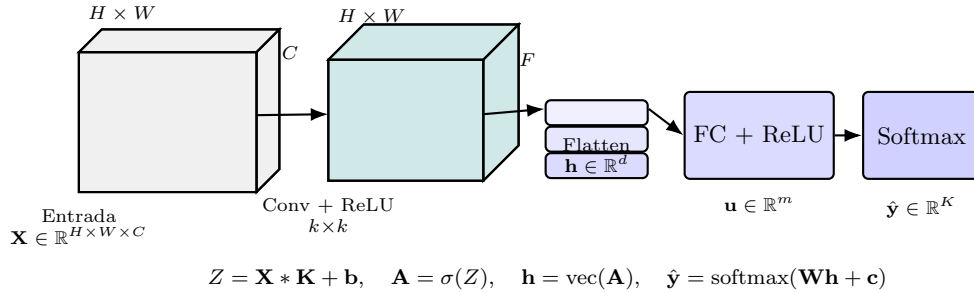


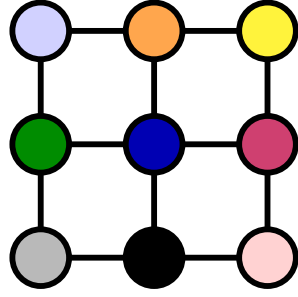
Figura 3.10: Arquitectura CNN simple ilustrada con volúmenes 3D. La red consta de una capa convolucional seguida de una capa completamente conectada y una capa de salida Softmax.

3.3. Redes Neuronales Gráficas

Las **redes neuronales gráficas** (GNN) utilizan atributos de nodos y relaciones de adyacencia para construir representaciones aprendidas. Se retoman la gráfica simple, el vecindario $N_G(v)$ y la matriz de adyacencia del Capítulo 2 (Definiciones 23, 25 y 28). Estas nociones permiten precisar qué información recibe cada nodo sin volver a definir la estructura combinatoria.

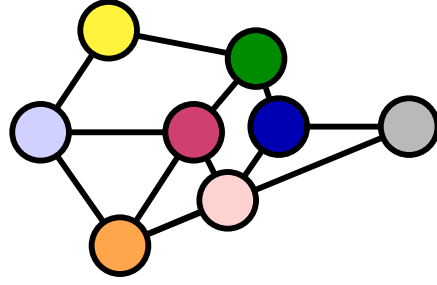
Para una gráfica de orden ν_G , se fija una enumeración $v_1, \dots, v_{\nu_G}$ y una matriz de atributos $\mathbf{X} \in \mathbb{R}^{\nu_G \times d_0}$. Cada fila corresponde al mismo nodo que la fila y columna respectivas de la matriz de adyacencia $\mathbf{A}$. En la representación de superficies del Capítulo 4, las coordenadas pueden servir como atributos iniciales. Los encajes que produce la red son representaciones aprendidas y no necesariamente posiciones en el espacio ambiente.

Seguendo [7], se distinguen la codificación de los nodos, el intercambio de información local y la obtención de una predicción. La clasificación de una superficie completa requiere además combinar las representaciones de sus nodos en una representación global.



CNN

En espacio euclidiano



GNN

En espacio no euclidiano

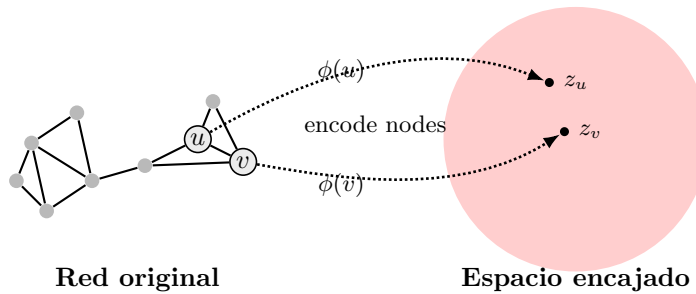
3.3.1. Codificación y predicción

Se utilizan los términos *encoder* y *decoder* para separar la construcción de una representación de su transformación en una predicción. Aquí se considera la tarea de clasificación; no se presupone que el segundo reconstruya la gráfica ni sea una inversa del primero.

Definición 62 (Encoder). *Fijadas una gráfica $G = (V, E)$, sus características iniciales y los parámetros del modelo, un encoder o codificador de nodos es una aplicación*

$$\phi : V \longrightarrow \mathbb{R}^d, \quad \phi(v) = z_v.$$

El vector z_v es el encaje del nodo v . En los codificadores basados en propagación de mensajes, esta representación depende de los atributos de v y de la información que se agrega a través de las adyacencias. La dependencia de la gráfica, los atributos y los parámetros queda implícita al escribir $\phi(v)$.



Los encajes de los nodos se organizan en una matriz respetando la enumeración usada para $\mathbf{X}$ y $\mathbf{A}$. Esto permite expresar varias capas de propagación mediante operaciones matriciales.

Definición 63 (Matriz de encajes). Sea $G = (V, E)$ una gráfica, con $|V| = \nu_G$. La matriz de encajes es una matriz

$$\mathbf{Z} \in \mathbb{R}^{\nu_G \times d}$$

cuyas filas corresponden a los encajes de los vértices. Para cada $v \in V$, el vector de encaje asociado se denota por

$$\phi(v) = \mathbf{Z}[v],$$

donde $\mathbf{Z}[v]$ representa la fila de $\mathbf{Z}$ correspondiente al vértice v , y $\phi(v) = z_v \in \mathbb{R}^d$ es el vector de encaje del nodo v .

Cada fila contiene información de un nodo. La matriz completa aún no es un único vector que represente la gráfica; para obtenerlo se introducirá una agregación global.

Una vez obtenidos los encajes, una función de predicción los transforma en salidas acordes con la tarea.

Definición 64 (Decoder). Un decoder o función de predicción es una aplicación

$$\psi : \mathbb{R}^d \longrightarrow \hat{\mathcal{Y}}$$

que transforma un encaje en una predicción. Para una tarea por nodo se escribe $\hat{y}_v = \psi(z_v)$. Si se utiliza un encaje global de la gráfica, la salida corresponde a la gráfica completa. El espacio $\hat{\mathcal{Y}}$ es el espacio de predicciones de la Definición 44.

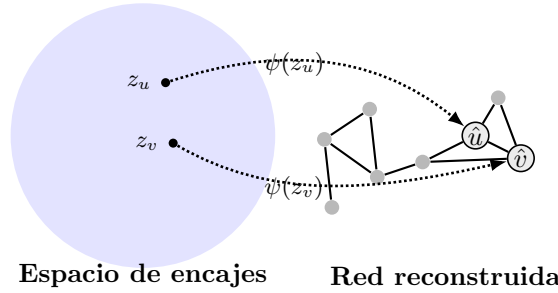


Figura 3.11: Predicción por nodo a partir de un encaje. Para clasificar la gráfica completa se requiere una representación global previa.

Estas funciones separan representación y predicción. La siguiente construcción especifica cómo obtener los encajes mediante información de los vecindarios.

3.3.2. Propagación de Mensajes Neuronales

La propagación de mensajes actualiza el estado de cada nodo a partir de su estado anterior y de los estados de sus vecinos. Se fija $h_v^{(0)} = x_v$, donde x_v es el vector de atributos iniciales de v . Los estados de una capa se calculan utilizando los de la capa anterior; no se mezclan actualizaciones de la misma iteración.

Definición 65 (Propagación de mensajes neuronales). Sea $G = (V, E)$ y, para $t \geq 1$, sean $h_u^{(t-1)}$ los estados de los nodos en la capa anterior. Una forma básica de mensaje agregado en el nodo v es

$$m_v^{(t)} = \mathcal{A}((h_u^{(t-1)})_{u \in N_G(v)}),$$

donde $\mathcal{A}$ combina una familia de vectores sin depender de su orden. La familia conserva multiplicidades: dos vecinos con el mismo estado siguen aportando dos elementos. Ejemplos son la suma, el promedio y el máximo componente a componente. Para un vecindario vacío debe fijarse una convención; en la suma se utiliza el vector cero.

El mensaje agregado se combina con el estado previo del nodo central mediante una función de actualización.

Definición 66 (Actualización de estados neuronales). Sea $h_v^{(t-1)}$ el encaje oculto del nodo v en la capa anterior $(t-1)$, y $m_v^{(t)}$ el mensaje agregado en la capa t . Entonces, la actualización del estado del nodo está dada por:

$$h_v^{(t)} = \mathcal{U}(h_v^{(t-1)}, m_v^{(t)}),$$

donde $\mathcal{U}$ es una función de actualización, típicamente implementada mediante una red neuronal, por ejemplo, una capa lineal seguida de una función de activación (como ReLU).

Tras K iteraciones, el estado final define el encaje del nodo:

$$z_u = h_u^{(K)}.$$

La Figura 3.12 reúne ambas operaciones. Si cada capa utiliza únicamente vecinos inmediatos, tras K capas el estado de un nodo puede depender de atributos situados a lo más a K aristas de distancia. Esto expresa un alcance combinatorio, no una garantía de recuperar invariantes topológicos de la superficie.

Iteración t : Propagación local

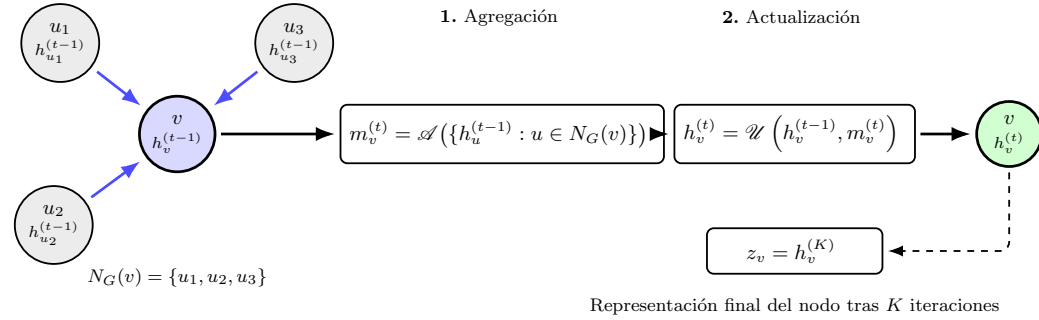


Figura 3.12: Esquema del mecanismo de propagación de información en una red neuronal sobre gráficas. En la iteración t , cada nodo v agrega las representaciones de sus vecinos mediante una función invariante al orden $\mathcal{A}$, obteniendo $m_v^{(t)}$. Posteriormente, su representación se actualiza mediante $\mathcal{U}$, combinando el estado previo con la información agregada. Tras K iteraciones, se obtiene la representación final $z_u = h_u^{(K)}$, que captura información de los atributos y las adyacencias.

3.3.3. La GNN fundamental

La agregación anterior puede ampliarse transformando primero cada par de estados del nodo central y de un vecino. Esta formulación distingue la función que produce mensajes de la que actualiza el estado, y sirve como antecedente de las operaciones locales presentadas en el Capítulo 5.

Definición 67 (Modelo GNN fundamental). Sean $x_u \in \mathbb{R}^{d_0}$ los atributos del nodo u , $\mathcal{M} : \mathbb{R}^{d_0} \times \mathbb{R}^{d_0} \rightarrow \mathbb{R}^r$ una función de mensajes y $\mathcal{U} : \mathbb{R}^{d_0} \times \mathbb{R}^r \rightarrow \mathbb{R}^{d_1}$ una función de actualización. Una capa de propagación puede escribirse como

$$h_u = \mathcal{U} \left(x_u, \bigoplus_{v \in N_G(u)} \mathcal{M}(x_u, x_v) \right),$$

donde $\bigoplus$ agrega, sin depender del orden, la familia de mensajes indexada por los vecinos. Esta familia conserva las multiplicidades. Las funciones $\mathcal{M}$ y $\mathcal{U}$ comparten sus parámetros entre los nodos de la capa.

La operación comprende tres pasos: producir un mensaje por vecino, agregar esos mensajes y actualizar el nodo central. La notación $\mathcal{M}$ y $\mathcal{U}$ distingue estos componentes del codificador ϕ y del predictor ψ . En capas posteriores se sustituyen los atributos iniciales por los estados de la capa anterior. Cuando se usan promedio o máximo, debe especificarse el tratamiento de nodos aislados o incluirse al propio nodo en la agregación.

3.3.4. Arquitectura y simetrías de una GNN

La forma matricial organiza simultáneamente las actualizaciones de todos los nodos y permite precisar el papel de la entrada que se modifica en el Capítulo 4.

Definición 68 (Arquitectura de una GNN). Sean $\mathbf{A} \in \mathbb{R}^{\nu_G \times \nu_G}$ la matriz de adyacencia y $\mathbf{H}_0 = \mathbf{X} \in \mathbb{R}^{\nu_G \times d_0}$ la matriz de características iniciales. Una GNN con K capas de propagación se expresa como

$$\mathbf{H}_{t+1} = \mathbf{F}(\mathbf{H}_t, \mathbf{A}, \Phi_t), \quad t = 0, \dots, K-1,$$

donde $\mathbf{H}_t \in \mathbb{R}^{\nu_G \times d_t}$, Φ_t contiene los parámetros de la capa y $\mathbf{F}$ combina mensajes y actualización con dimensiones compatibles. La notación $\mathbf{F}$ representa la operación de cada capa, cuyos parámetros y dimensiones pueden variar con t . La matriz final $\mathbf{H}_K$ es la matriz de encajes de la Definición 63.

Esta formulación utiliza una adyacencia fija. En modelos que reconstruyen sus conexiones durante las capas se requiere una matriz $\mathbf{A}_t$ dependiente del estado; tal es la distinción que se retomará al presentar DGCNN. Cambiar la matriz de entrada mediante muestreo tampoco equivale a aprender nuevas características: son operaciones diferentes.

La equivarianza de la Definición 54 se especializa ahora a la enumeración de nodos. Al cambiar esa enumeración, deben permutarse de forma compatible tanto las características como la adyacencia.

Definición 69 (Equivarianza a permutaciones). Sean $\mathbf{H} \in \mathbb{R}^{\nu_G \times d}$, $\mathbf{A} \in \mathbb{R}^{\nu_G \times \nu_G}$ y $\mathbf{P} \in \mathbb{R}^{\nu_G \times \nu_G}$ una matriz de permutación. Una capa $\mathbf{F}$ es **equivariante a permutaciones** si, para toda entrada admisible y toda $\mathbf{P}$,

$$\mathbf{F}(\mathbf{P}\mathbf{H}, \mathbf{P}\mathbf{A}\mathbf{P}^T, \Phi) = \mathbf{P}\mathbf{F}(\mathbf{H}, \mathbf{A}, \Phi).$$

La multiplicación por $\mathbf{P}$ a la izquierda reordena las filas, que corresponden a los nodos. La salida se reordena de la misma manera; no permanece necesariamente igual como matriz.

La propagación con funciones compartidas y agregación independiente del orden respeta este cambio de enumeración: cada nodo recibe la misma familia de mensajes, sólo reindexada. Esta propiedad de las capas no se transfiere automáticamente al procedimiento de reducción del Capítulo 4, cuyas decisiones también dependen del recorrido y de los índices.

Para clasificar una gráfica completa se utiliza una agregación global $\mathcal{R}$ sobre los encajes, seguida de una función predictora:

$$z_G = \mathcal{R}((z_v)_{v \in V}), \quad \hat{y}_G = \psi(z_G).$$

Si $\mathcal{R}$ es independiente del orden, por ejemplo una suma o un máximo componente a componente, y los encajes son equivariantes, la predicción es invariante a la enumeración. Así se distingue la representación por nodo de la representación global necesaria para clasificar una superficie.

3.4. Redes gráficas convolucionales (GCN)

Una GCN es un caso particular de GNN que utiliza una agregación convolucional sobre la gráfica. Para concretar esta diferencia, se considera la agregación normalizada por grados descrita en [7]. Sean

$$\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}, \quad \tilde{\mathbf{D}}_{ii} = \sum_j \tilde{\mathbf{A}}_{ij},$$

donde $\mathbf{I}$ es la identidad y $\tilde{\mathbf{D}}$ es diagonal. La identidad incorpora el estado propio del nodo en la propagación; $\tilde{\mathbf{A}}$ no es la matriz de adyacencia de la gráfica simple original. Esta construcción usa el grado de la Definición 26 y permite definir una capa mediante

$$\mathbf{H}_{t+1} = \sigma\left(\tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2} \mathbf{H}_t \mathbf{W}_t\right).$$

Aquí $\mathbf{W}_t \in \mathbb{R}^{d_t \times d_{t+1}}$ transforma las características, la matriz normalizada combina vecinos y σ actúa componente a componente. Los pesos multiplican a la derecha porque cada nodo ocupa una fila, a diferencia de la convención de vectores columna usada en las capas densas. La Figura 3.13 ilustra el intercambio local de información.

Los fundamentos anteriores permiten distinguir las entradas y las operaciones de los modelos estudiados en el Capítulo 5. PointNet procesa conjuntos de puntos mediante transformaciones compartidas y agregación simétrica [?]; no requiere propagación sobre una matriz de adyacencia. Las GNN emplean relaciones entre nodos y las GCN constituyen una familia particular dentro de ellas. Por su parte, FNO se presenta como un operador neuronal, no como una GCN. Las arquitecturas específicas se desarrollarán allí, sin identificar

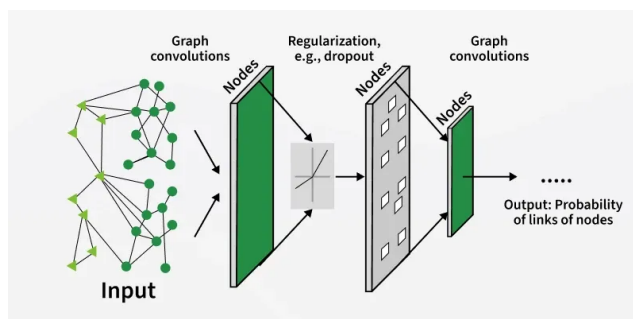


Figura 3.13: Ilustración de la agregación local de información sobre una gráfica.

automáticamente todos los modelos sobre datos geométricos con redes gráficas.

El Capítulo 4 utiliza estas distinciones para transformar las coordenadas y adyacencias de entrada antes del aprendizaje. La evaluación posterior examina el efecto de esa representación en la clasificación; ni la equivarianza de una capa ni una predicción acertada demuestran, por sí solas, que la reducción conserve la topología de la superficie.

Capítulo 4

Construcción de algoritmo de muestreo

El Capítulo 2 estableció los fundamentos para representar superficies mediante complejos simpliciales, mientras que el Capítulo 3 introdujo el uso de las relaciones de adyacencia en modelos neuronales sobre gráficas. A partir de la Definición 17 y del mecanismo de propagación de mensajes de la Definición 65, este capítulo desarrolla la metodología implementada para reducir el número de nodos de la representación computacional de una superficie triangulada.

La elección de la representación de entrada forma parte del estudio de modelos de aprendizaje profundo sobre datos geométricos [1,6]. En esta tesis, EuLearn [5] proporciona el contexto de datos y clasificación de superficies al que se aplica el procedimiento. El muestreo transforma la representación de cada superficie antes de su procesamiento por los modelos; no constituye el entrenamiento de la red ni la selección de observaciones del conjunto de entrenamiento de la Definición 43. La descripción del método implementado se distingue de los resultados reportados por los autores de EuLearn y de los experimentos de esta tesis, que se presentan en el Capítulo 5.

La exposición sigue la cadena superficie, complejo simplicial, gráfica original, árbol auxiliar, proceso de reducción y gráfica reducida. Primero se identifican las coordenadas y adyacencias utilizadas como entrada; después se describen la construcción de un árbol generador mínimo, el recorrido en profundidad, la selección espacial de candidatos y la reasignación de conexiones. Finalmente, estos componentes se integran en el método general y se analiza su costo computacional tomando como referencia la implementación del Apéndice B.

El propósito de la reasignación es incorporar información de las conexiones al disminuir el número de nodos. Como el procedimiento actúa sobre coordenadas y adyacencias, sin actualizar las caras de la triangulación, se distingue esta reducción combinatoria de la conservación de propiedades topológicas de la superficie.

4.1. Construcción de superficies a partir de símlices

La construcción descrita en este trabajo parte de curvas de Lissajous en $\mathbb{R}^3$ y de superficies asociadas a su engrosamiento. Una curva es un objeto unidimensional; una región obtenida al engrosarla puede tener volumen, mientras que la superficie que delimita dicha región es bidimensional cuando satisface las condiciones de la Definición 16. Por tanto, estar situada en $\mathbb{R}^3$ no convierte a la superficie en un objeto de dimensión tres. La implementación de muestreo del Apéndice B recibe coordenadas y adyacencias ya construidas: no especifica la parametrización, el engrosamiento ni la triangulación de estas superficies.

La representación computacional considerada es una malla triangular, que puede al-

macenarse en formato STL como una colección de triángulos. Al identificar los vértices compartidos, cada triángulo determina tres aristas. Esta descripción establece la representación de entrada, sin atribuir al código de muestreo operaciones de construcción de mallas que no aparecen en él.

Retomando la Definición 17, se utiliza la notación computacional $K = (V, E, F)$ para registrar los vértices, las aristas y los 2-símplices o caras triangulares. Esta terna organiza los símplices por dimensión; no sustituye la definición abstracta de complejo como familia cerrada bajo la toma de caras. En particular, cada cara triangular incluye sus aristas y vértices en el complejo. La Figura 4.1 ilustra esta correspondencia.

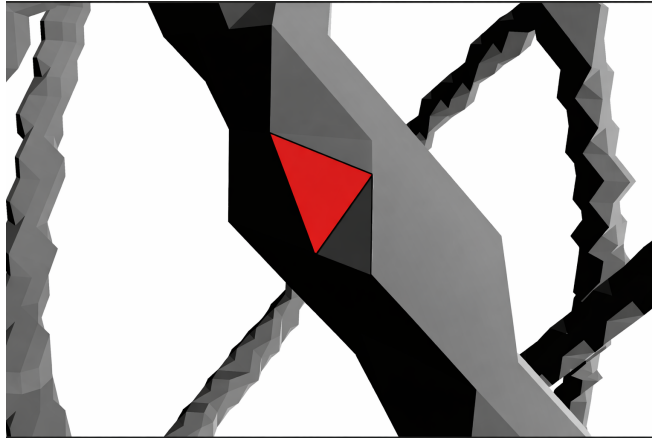


Figura 4.1: Identificación de una cara triangular en la malla. La región roja representa un 2-símplice; sus lados y vértices corresponden a los símplices de dimensiones uno y cero.

Para una triangulación finita de una superficie, la característica de Euler se calcula mediante

$$\chi = |V| - |E| + |F|.$$

La información de F es indispensable para interpretar esta expresión como un invariante de la superficie triangulada. El algoritmo estudiado no recibe ni actualiza F ; por ello, su salida no determina automáticamente una nueva triangulación y no permite concluir que se conserve la característica de Euler o el género.

Una malla finita tiene realización geométrica compacta, en el sentido de la Definición 9, pero no toda malla triangular representa una superficie. Para una superficie triangulada sin borde deben verificarse, entre otras condiciones, que cada arista pertenezca a dos caras y que cada vértice admita un entorno abierto homeomorfo a un disco abierto. La ausencia de borde como variedad no debe confundirse con la frontera de un subconjunto del espacio ambiente de la Definición 5. En este contexto, una superficie cerrada es compacta y sin borde; estas propiedades y su género requieren comprobaciones que no se deducen de la Figura 4.2 ni son realizadas por la función de muestreo.

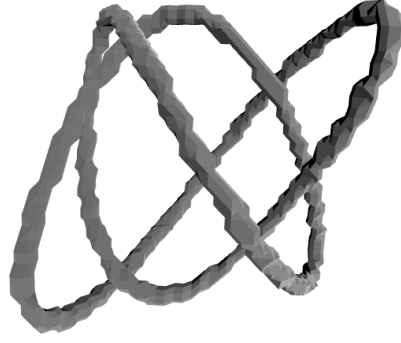


Figura 4.2: Visualización de una malla triangular asociada a la construcción basada en Lissajous. La figura ilustra su disposición espacial; por sí sola no verifica la ausencia de borde, el género ni la equivalencia topológica con otra superficie.

4.2. Transformación de superficie a gráfica

A partir de K , se considera su 1-esqueleto $G = (V, E)$, conforme a la Definición 23. Se mantienen los mismos vértices y aristas y se omite F en la entrada del algoritmo. Este paso conserva la adyacencia entre los vértices originales, pero deja de registrar explícitamente las caras; todavía no reduce el número de nodos.

A cada vértice $v \in V$ se le asocia una aplicación de posición

$$\text{pos}(v) = (x_v, y_v, z_v) \in \mathbb{R}^3.$$

Las filas del arreglo $x \in \mathbb{R}^{\nu_G \times 3}$ contienen estas coordenadas, donde ν_G es el orden de la gráfica según la Definición 24. Junto con la matriz de adyacencia A de la Definición 28, determinan la entrada computacional del método. Se supone que la fila de coordenadas y la fila y columna de adyacencias con el mismo índice corresponden al mismo nodo.

En la arquitectura de la Definición 68, las coordenadas pueden utilizarse como características iniciales, mientras que A determina las conexiones para la propagación de mensajes. Estas posiciones geométricas no deben confundirse con la matriz de encajes aprendidos de la Definición 63. La Figura 4.3 muestra conjuntamente la posición de los nodos y su adyacencia; si se omiten las aristas, queda únicamente la nube de puntos.



Figura 4.3: Representación espacial del 1-esqueleto de la malla. Los nodos azules conservan las coordenadas de los vértices originales y los segmentos representan sus aristas; las caras no forman parte de esta representación de entrada.

4.3. Árbol generador mínimo y recorrido auxiliar

Antes de seleccionar nodos para su eliminación, se construye una estructura auxiliar que organiza el recorrido inicial. Un árbol generador es una subgráfica generadora y, por tanto, conserva todos los vértices de G (Definiciones 38 y 31). Para una gráfica conexa existe tal árbol, por el Teorema 4, y contiene $\nu_G - 1$ aristas según el Teorema 3. Construirlo no elimina nodos: la reducción ocurre en etapas posteriores.

La Definición 39 introduce la minimización de la suma de pesos. La implementación pasa directamente la matriz binaria A a `minimum_spanning_tree`, de SciPy, que utiliza Kruskal. En consecuencia, todas las aristas presentes tienen peso uno y cualquier árbol generador tiene peso $\nu_G - 1$; no se emplean distancias euclidianas como pesos ni se establece una prioridad geométrica entre aristas. Los pesos distintos de la Figura 4.4 ilustran el problema general ponderado, no los datos que recibe esta implementación.

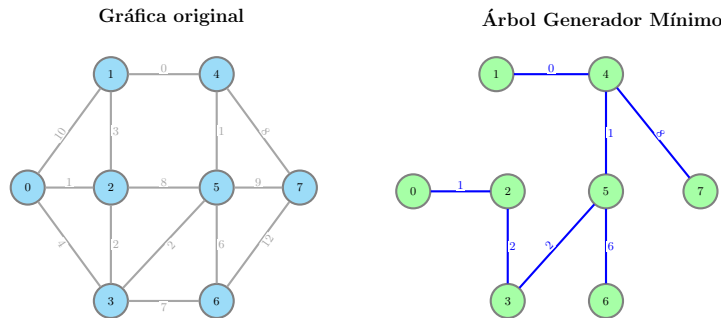


Figura 4.4: Ejemplo ilustrativo de un árbol generador mínimo en una gráfica ponderada. Ambos objetos contienen los mismos vértices. Los pesos de este dibujo, incluido el cero, no representan la matriz binaria usada por el código, donde cero indica ausencia de arista.

El Algoritmo 3 presenta la formulación general. La estructura *Union-Find* mantiene una partición de los vértices: `FIND` identifica el componente al que pertenece un vértice y `UNION` reúne dos componentes. Así se decide si añadir una arista formaría un ciclo.

Algorithm 3 Algoritmo de Kruskal para el árbol generador mínimo

Require: Gráfica conexa no dirigida $G = (V, E)$ con pesos en las aristas**Ensure:** Árbol generador mínimo

```

1: Inicializar  $MST \leftarrow \emptyset$  ▷ Conjunto vacío para el MST
2: Ordenar las aristas  $E$  en orden creciente de pesos
3: Inicializar estructura Union-Find para los vértices  $V$ 
4: for cada arista  $(u, v) \in E$  en orden creciente de peso do
5:   if  $\text{FIND}(u) \neq \text{FIND}(v)$  then ▷ Si u y v no están en el mismo componente
6:      $MST \leftarrow MST \cup \{(u, v)\}$  ▷ Añadir arista al MST
7:      $\text{UNION}(u, v)$  ▷ Unir los componentes de u y v
8:   end if
9:   if Número de aristas en  $MST = |V| - 1$  then ▷ MST completo
10:    break
11:   end if
12: end for
13: return  $MST$  ▷ Devolver el árbol generador mínimo

```

La matriz devuelta por la rutina se simetriza mediante $A_s = A_{\text{mst}} + A_{\text{mst}}^T$. Sobre A_s , almacenada como `mst`, se aplica búsqueda en profundidad (DFS). Para una entrada conexa, el recorrido visita todos los nodos una vez; la matriz original A se conserva por separado para seleccionar candidatos. Si la entrada es desconexa, la rutina de árbol produce un bosque y el DFS iniciado únicamente en el nodo cero recorre sólo su componente.

El orden depende del vértice inicial y del orden de exploración de los vecinos. En el código se inicia en cero y `np.where` enumera los vecinos por índice creciente. Por tanto, este recorrido no constituye una selección libre de sesgos ni una garantía de equivarianza a permutaciones en el sentido de la Definición 69.

Algorithm 4 Recorrido en profundidad del árbol auxiliar

Require: Matriz simétrica mst , nodo actual $node$, arreglo $visited$, lista $order$ **Ensure:** Se añaden a $order$ los nodos alcanzables todavía no visitados

```

1: function DFS_ORDER( $node, visited, order$ )
2:    $visited[node] \leftarrow \text{True}$ ; añadir  $node$  a  $order$ 
3:   for all  $neighbor$  en índices crecientes tales que  $mst[node, neighbor] = 1$  do
4:     if  $visited[neighbor] = \text{False}$  then
5:       DFS_ORDER( $neighbor, visited, order$ )
6:     end if
7:   end for
8: end function

```

El Algoritmo 4 determina únicamente el orden inicial. Una vez que se han marcado nodos para eliminación, el método general vuelve a ordenar los nodos pendientes mediante sus distancias al centroide de los que permanecen.

4.4. Muestreo por esferas

El siguiente componente combina la adyacencia de G con un criterio espacial. El vecindario $N_G(v)$ de la Definición 25 contiene vértices, mientras que una bola en $\mathbb{R}^3$ contiene posiciones. Esta distinción permite relacionar la selección con los vecinos utilizados en la propagación de mensajes (Definición 65) sin identificar adyacencia y cercanía euclidiana.

Para un nodo central v , el código genera $s = \text{esfera_size}$ puntos auxiliares $q_1, \dots, q_s$ dentro de la bola unitaria centrada en $\text{pos}(v)$. Estos puntos no se incorporan como vértices de la malla. Con $\rho = \text{alcance}$, la región de búsqueda es

$$\bigcup_{j=1}^s \{p \in \mathbb{R}^3 : \|p - q_j\|_2 \leq \rho\}.$$

Por ello, el criterio efectivo para un candidato u es

$$u \in N_G(v), \quad \min_{1 \leq j \leq s} \|\text{pos}(u) - q_j\|_2 \leq \rho.$$

No se trata simplemente de tomar todos los vértices contenidos en una bola de radio ρ centrada en v . Además, $N_G(v)$ se consulta siempre en la matriz original, no en la matriz modificada durante la reducción. Los nodos ya marcados para eliminación se excluyen después.

Para justificar la generación aleatoria, conviene distinguir el método polar para variables normales [9] de la transformación empleada para obtener direcciones y puntos en una bola. Ambos parten del rechazo de puntos fuera del disco unitario; la función del apéndice no genera variables normales como paso intermedio.

Algorithm 5 Método polar para desviaciones normales

- 1: **repeat**
 - 2: Generar V_1, V_2 independientes y uniformes en $(-1, 1)$
 - 3: $S \leftarrow V_1^2 + V_2^2$
 - 4: **until** $0 < S < 1$
 - 5: $X_1 \leftarrow V_1 \sqrt{-2 \ln(S)/S}$; $X_2 \leftarrow V_2 \sqrt{-2 \ln(S)/S}$
 - 6: **return** X_1, X_2
-

El Algoritmo 5 produce dos variables normales estándar independientes, como se comprueba a continuación.

Demostración. Condicionado a $0 < S < 1$, el par (V_1, V_2) es uniforme en el disco unitario, salvo el origen, de probabilidad cero. En coordenadas polares, $V_1 = R \cos \theta$ y $V_2 = R \sin \theta$, la densidad conjunta es r/π para $0 < r < 1$ y $0 \leq \theta < 2\pi$. Por tanto, $S = R^2$ es uniforme en $(0, 1)$ e independiente de θ , que es uniforme en $[0, 2\pi)$.

Si $R' = \sqrt{-2 \ln S}$, entonces, para $r \geq 0$,

$$\mathbb{P}(R' \leq r) = \mathbb{P}(S \geq e^{-r^2/2}) = 1 - e^{-r^2/2}.$$

Su densidad es $re^{-r^2/2}$, y R' sigue siendo independiente de θ . Al transformar (R', θ) a

$(X_1, X_2) = (R' \cos \theta, R' \sin \theta)$, el jacobiano polar da la densidad

$$f_{X_1, X_2}(x_1, x_2) = \frac{1}{2\pi} e^{-(x_1^2 + x_2^2)/2} = \frac{e^{-x_1^2/2}}{\sqrt{2\pi}} \frac{e^{-x_2^2/2}}{\sqrt{2\pi}}.$$

La factorización prueba la afirmación. $\square$

Para construir direcciones en la esfera se utiliza el mismo par aceptado (V_1, V_2) , pero una transformación distinta. Se toma $Z = 2S - 1$ y el ángulo polar ϕ de dicho par. Las otras coordenadas satisfacen

$$X = \sqrt{1 - Z^2} \cos \phi, \quad Y = \sqrt{1 - Z^2} \sin \phi. \quad (4.1)$$

Como $\cos \phi = V_1/\sqrt{S}$ y $\sin \phi = V_2/\sqrt{S}$, al sustituir en (4.1) se obtiene

$$\sqrt{\frac{1 - (2S - 1)^2}{S}} = \sqrt{\frac{4S - 4S^2}{S}} = 2\sqrt{1 - S}.$$

Así, la dirección utilizada por el código es

$$(X, Y, Z) = (2V_1\sqrt{1 - S}, 2V_2\sqrt{1 - S}, 2S - 1),$$

y cumple

$$X^2 + Y^2 + Z^2 = 4S(1 - S) + (2S - 1)^2 = 1.$$

La pertenencia a la esfera no basta para probar uniformidad. En este caso, Z es uniforme en $(-1, 1)$ e independiente de ϕ , uniforme en $[0, 2\pi)$; como el elemento de área de la esfera unitaria en estas coordenadas es $d\phi dZ$, la dirección es uniforme respecto del área. La Figura 4.5 ilustra direcciones sobre la esfera, no la etapa radial posterior.

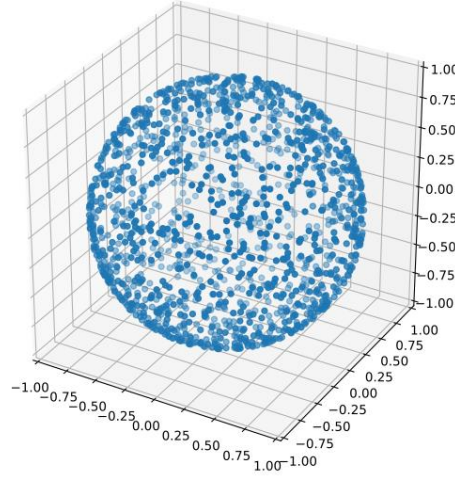


Figura 4.5: Ilustración de puntos sobre la esfera unitaria. La uniformidad de las direcciones se justifica mediante su distribución, no por la apariencia de la nube de puntos.

Para obtener puntos uniformes respecto del volumen de la bola unitaria se genera, independientemente, $U \sim \mathcal{U}(0, 1)$ y se toma $r = U^{1/3}$. Entonces $\mathbb{P}(r \leq t) = t^3$ para $0 \leq t \leq 1$, que coincide con la proporción de volumen de una bola de radio t . El punto final, con centro (h, k, l) , es

$$q = (h, k, l) + r(X, Y, Z).$$

La traslación se realiza después del escalamiento radial. El radio de esta bola es uno; **alcance** se usa después en la selección de candidatos y no modifica la distribución de los puntos auxiliares.

Algorithm 6 Generación de puntos uniformes dentro de una bola unitaria

Require: Centro (h, k, l) y entero positivo num_puntos **Ensure:** Arreglo de num_puntos puntos auxiliares

```

1: function GENERAPUNTOSUNIFORMES( $h, k, l, num\_puntos$ )
2:    $puntos \leftarrow []$ 
3:   for  $i \leftarrow 1$  to  $num\_puntos$  do
4:     repeat
5:       Generar  $V_1, V_2$  independientes, uniformes en  $(-1, 1)$ 
6:        $S \leftarrow V_1^2 + V_2^2$ 
7:     until  $0 < S < 1$ 
8:      $a \leftarrow \sqrt{1 - S}$ 
9:      $dx \leftarrow 2V_1a; dy \leftarrow 2V_2a; dz \leftarrow 2S - 1$ 
10:    Generar  $U \sim \mathcal{U}(0, 1)$  independiente;  $r \leftarrow U^{1/3}$ 
11:    Añadir  $[h + r\,dx, k + r\,dy, l + r\,dz]$  a  $puntos$ 
12:  end for
13:  return array( $puntos$ )
14: end function

```

La Figura 4.6 resume la generación y el uso posterior de los puntos del Algoritmo 6.

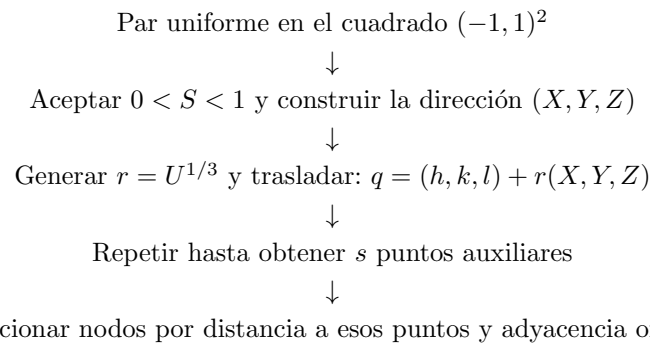


Figura 4.6: Flujo de generación de puntos auxiliares y selección de candidatos. Los puntos generados no se añaden a la gráfica.

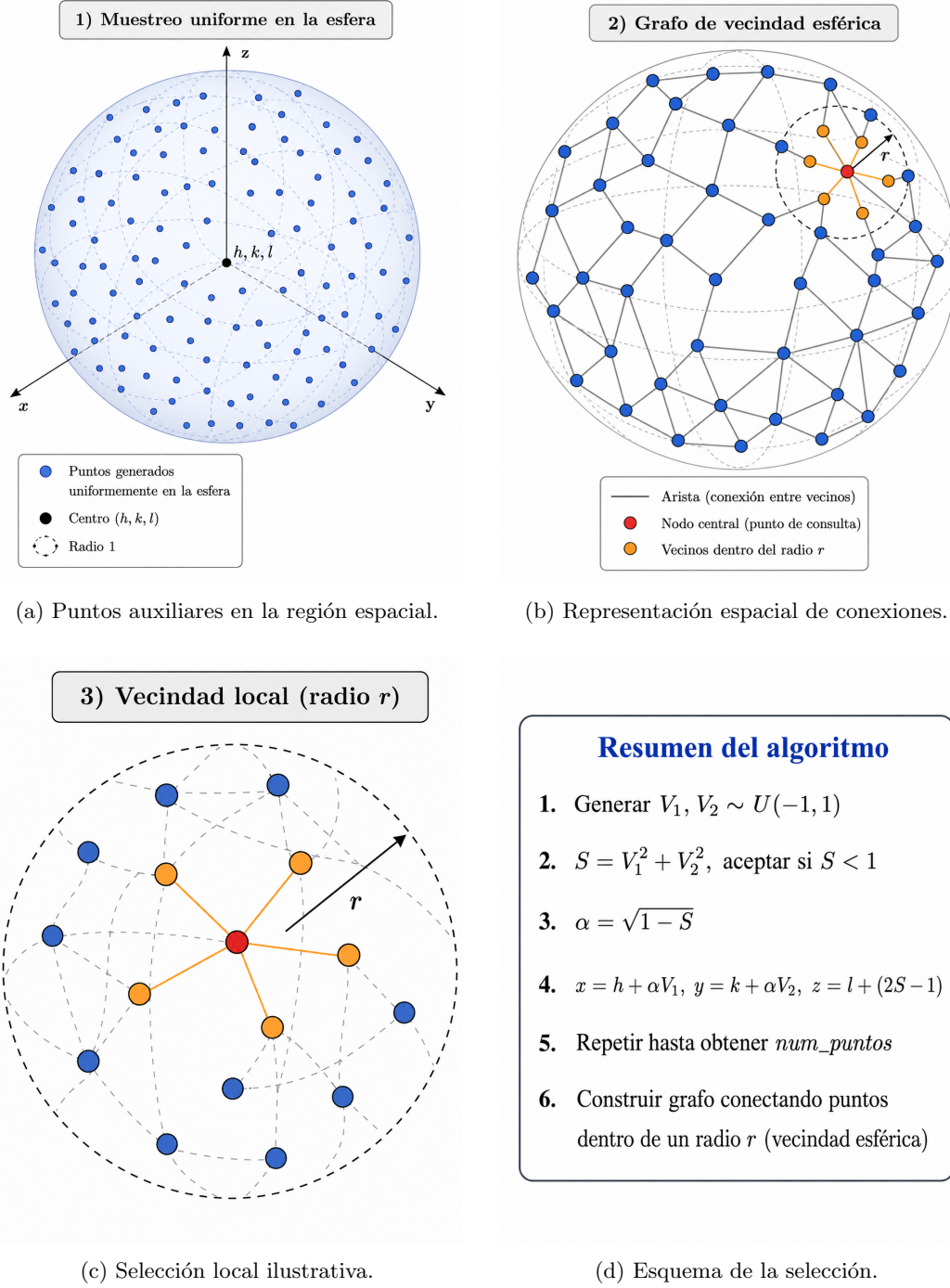


Figura 4.7: Ilustración de la selección espacial. La condición implementada combina la distancia a puntos auxiliares con la adyacencia original; el dibujo no representa una reconstrucción de las caras de la malla.

La Figura 4.7 ofrece una representación esquemática de esta etapa. Una vez seleccionados los candidatos, es necesario precisar qué conexiones modifica realmente la implementación.

4.5. Reasignación de adyacencias

El procedimiento del Apéndice B acumula conexiones en el nodo central de la iteración antes de aislar los nodos seleccionados. Esta operación no equivale a conectar entre sí todos los vecinos de un vértice eliminado, ni a transferir únicamente su fila de adyacencias. Las operaciones de transferencia e aislamiento están implementadas directamente dentro de `sampling`.

Se mantiene la matriz de adyacencia de la Definición 28; la única convención adicional es la indexación de Python, $i, j \in \{0, \dots, \nu_G - 1\}$. Durante el proceso se conserva A sin cambios y se utiliza una copia numérica B , correspondiente a `new_adj`, para acumular las actualizaciones. Sea D el conjunto de índices ya marcados para eliminación y sea idx el nodo central.

Para cada candidato n , en el orden de `neigs`, se calcula

$$J_n = \{j : B_{nj} = 1\} \setminus D, \quad b_t^{(n)} = \sum_{j \in J_n} B_{jt}, \quad 0 \leq t < \nu_G.$$

Si J_n es vacío no se realiza la transferencia para ese candidato. En caso contrario, se suma el vector $b^{(n)}$ a la fila idx y luego a la columna idx :

$$B_{idx,:} \leftarrow B_{idx,:} + b^{(n)}, \quad B_{:,idx} \leftarrow B_{:,idx} + b^{(n)}.$$

El vector se calcula antes de ambas sumas. La entrada diagonal $B_{idx,idx}$ recibe dos incrementos; en consecuencia, B es un acumulador intermedio, no necesariamente la matriz binaria de una gráfica simple. Las actualizaciones son sumas numéricas, no operaciones de disyunción lógica.

La prueba literal del código es $B_{nj} = 1$, no $B_{nj} > 0$. Después de acumular sumas pueden aparecer entradas mayores que uno, que no se incluyen en búsquedas posteriores de J_n . Por ello, sólo al inicio J_n coincide con el vecindario de n en la gráfica original, salvo los índices de D . La suma de filas recoge conexiones de los vecinos seleccionados, relacionadas inicialmente con recorridos de dos aristas desde n (Definición 33); no debe interpretarse como una regla invariable sobre vecindarios de una matriz binaria.

Las actualizaciones de distintos candidatos son secuenciales: cada una utiliza la matriz ya modificada por las anteriores. Los demás candidatos del mismo lote no se excluyen mediante D , pues éste se actualiza al finalizar el lote. Después se ponen a cero todas las filas y columnas de `neigs`. Esto aísla esos nodos en el sentido de la Definición 27; sus índices siguen ocupando posiciones en la matriz hasta la eliminación final de filas y columnas.

Ejemplo 9. Se retoma la gráfica de la Figura 2.5 del Capítulo 2, ahora con índices desde cero. Su matriz es

$$A = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \end{pmatrix}.$$

En particular, $A_{02} = 0$, $A_{23} = 1$ y $A_{54} = 1$. Si $B = A$, $D = \emptyset$, $idx = 1$ y el único candidato

es $n = 5$, entonces $J_5 = \{1, 2, 4\}$ y

$$b^{(5)} = B_{1,:} + B_{2,:} + B_{4,:} = (1, 1, 1, 2, 0, 3).$$

Después de las dos sumas y de aislar el índice cinco, se obtiene

$$B = \begin{pmatrix} 0 & 2 & 0 & 0 & 0 & 0 \\ 2 & 2 & 2 & 2 & 0 & 0 \\ 0 & 2 & 0 & 1 & 0 & 0 \\ 0 & 2 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

Si se termina el proceso en este punto, se elimina la última fila y columna, se reemplazan las entradas positivas por uno y se anula la diagonal. El resultado es

$$A_{\text{red}} = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}.$$

Se añade la arista entre los índices uno y tres, pero no se conectan entre sí todos los elementos de J_5 .

El Algoritmo 7 representa el bloque interno de transferencia y aislamiento. Su nombre es auxiliar para el pseudocódigo; no introduce una función adicional en Python.

Algorithm 7 Transferencia al nodo central y aislamiento de un lote

Require: Nodo central idx , candidatos $neigs$, acumulador B , eliminados D

Ensure: Acumulador actualizado, con los candidatos aislados

```

1: function TRANSFERIR AISLAR( $idx, neigs, B, D$ )
2:   for all  $n$  en  $neigs$ , en su orden actual do
3:      $J_n \leftarrow \{j : B_{nj} = 1\} \setminus D$ 
4:     if  $J_n \neq \emptyset$  then
5:        $b \leftarrow \sum_{j \in J_n} B_{j,:}$ 
6:        $B_{idx,:} \leftarrow B_{idx,:} + b$ 
7:        $B_{:,idx} \leftarrow B_{:,idx} + b$ 
8:     end if
9:   end for
10:   $B[neigs,:] \leftarrow 0$ ;  $B[:,neigs] \leftarrow 0$ 
11:  return  $B$ 
12: end function

```

Las Figuras 4.8 y 4.9 distinguen la acumulación, el aislamiento y la eliminación efectiva. La matriz final es simétrica y tiene diagonal nula, suponiendo aritmética sin desbordamiento. Como pueden añadirse aristas, la gráfica reducida no es necesariamente una subgráfica de

G , ni la subgráfica inducida por los nodos conservados (Definiciones 30 y 32). Tampoco se reconstruyen las caras del complejo.

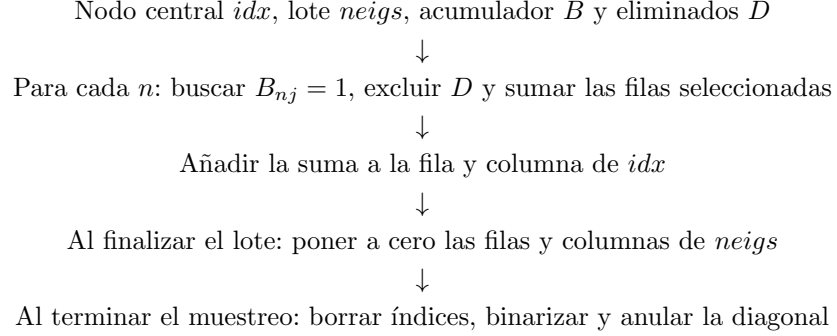


Figura 4.8: Flujo de la reasignación implementada. Las sumas numéricas preceden a la binarización final.

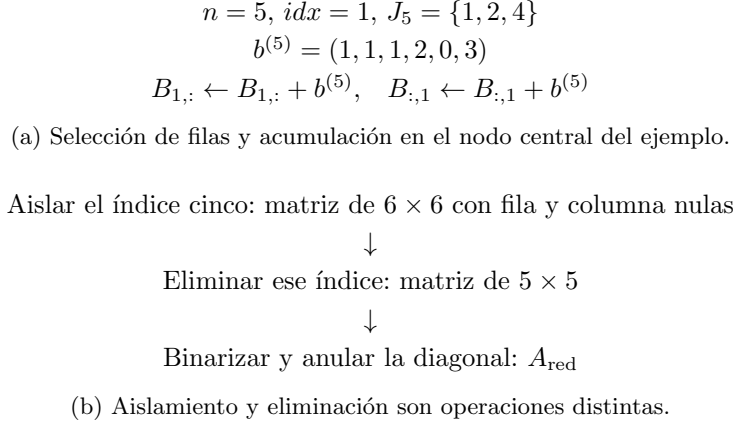


Figura 4.9: Etapas de la reasignación para el ejemplo. Se distingue el acumulador numérico de la matriz final de una gráfica simple.

4.6. Método general para la reducción de nodos

La función `sampling` recibe el arreglo de coordenadas x , la matriz binaria original `adjacency` = A y el número objetivo de nodos conservados `size`. Recibe además `alcance` ≥ 0 y el entero positivo `esfera_size`. Para la descripción se supone una gráfica simple conexa de orden $\nu_G \geq 1$, coordenadas alineadas con A y $1 \leq \text{size} \leq \nu_G$. El código no valida expresamente todas estas condiciones. Las salidas previstas son `new_x`, con las coordenadas conservadas, y `new_adj`, con la matriz reducida, sujetas a las limitaciones de implementación indicadas al final de esta sección.

Según la Definición 24, `size` representa el *orden objetivo*, no el tamaño de la gráfica, que es el número de aristas ε_G . El árbol del Algoritmo 3 y el recorrido del Algoritmo 4 inicializan la lista de nodos pendientes. El Algoritmo 6 determina los puntos auxiliares y el Algoritmo 7 expresa el bloque de modificación de conexiones.

Si D contiene los índices ya marcados, el centroide de los nodos que permanecen es

$$\frac{1}{\nu_G - |D|} \sum_{i \in \{0, \dots, \nu_G - 1\} \setminus D} x_i.$$

Después de la primera eliminación, la cola se ordena en sentido decreciente por la distancia individual de cada nodo a este centroide. No se calculan distancias promedio entre pares de nodos ni se demuestra que esta prioridad evite sesgos. La conversión de la cola a conjunto para retirar candidatos puede perder el orden previo; el criterio de desempate tampoco establece invariancia frente a un cambio de índices.

Los candidatos se obtienen mediante `np.intersect1d`, que los ordena por índice. Si hay más de los necesarios, se conservan los primeros hasta completar $\nu_G - |D| - \text{size}$ eliminaciones posibles. Esta truncación impide reducir el orden por debajo del objetivo, pero no garantiza alcanzarlo. El ciclo termina al alcanzar el objetivo o al agotarse la cola, siempre que no ocurra un error de ejecución. Cada iteración retira al menos su nodo central de la cola, aun si no selecciona vecinos; por ello hay a lo más ν_G iteraciones.

Algorithm 8 Método general de reducción: operaciones de **sampling**

Require: $x, A, \text{size}, \text{alcance}, \text{esfera_size}$, con las condiciones indicadas en el texto

Ensure: Coordenadas conservadas y matriz reducida, si la ejecución concluye

```

1:  $mst \leftarrow \text{minimum\_spanning\_tree}(A)$ , convertido a matriz entera
2:  $mst \leftarrow mst + mst^T$ ; inicializar  $visited$  y  $order$ 
3:  $\text{DFS\_ORDER}(0, visited, order)$ 
4:  $M \leftarrow \nu_G$ ;  $B \leftarrow \text{copy}(A)$ ;  $D \leftarrow \emptyset$ ;  $queue \leftarrow order$ 
5: while  $M - |D| > \text{size}$  and  $queue \neq []$  do
6:   if  $D \neq \emptyset$  then
7:     Calcular el centroide de  $\{x_i : i \notin D\}$ 
8:     Ordenar  $queue$  por distancia decreciente al centroide
9:   end if
10:   $idx \leftarrow queue.\text{pop}(0)$ ;  $(h, k, l) \leftarrow x[idx]$ 
11:   $Q \leftarrow \text{GENERAPUNTOSUNIFORMES}(h, k, l, \text{esfera\_size})$ 
12:   $P \leftarrow \{i : \min_{q \in Q} \|x_i - q\|_2 \leq \text{alcance}\}$ 
13:   $neigs \leftarrow \text{intersect1d}(P, \{i : A_{idx,i} = 1\})$ 
14:  Retirar de  $neigs$  los índices de  $D$ , conservando su orden
15:  Truncar  $neigs$  a los primeros  $M - |D| - \text{size}$  elementos, si es necesario
16:   $queue \leftarrow \text{list}(\text{set}(queue) \setminus \text{set}(neigs))$ 
17:   $B \leftarrow \text{TRANSFERIR AISLAR}(idx, neigs, B, D)$ 
18:   $D \leftarrow D \cup \text{set}(neigs)$ 
19: end while
20: Borrar de  $B$  las filas y columnas con índices de  $D$ 
21:  $B \leftarrow 1 \cdot (B > 0)$ ; anular su diagonal
22:  $index \leftarrow \text{list}(\text{set}(\{0, \dots, M - 1\}) \setminus D)$ 
23: return  $x[index, :], B$ 
```

El Algoritmo 8 conserva las operaciones y el orden de actualización del listado Python. Hay dos limitaciones concretas que deben considerarse al interpretar sus salidas. Primero,

si no hay candidatos, `np.array([n for n in neigs if n not in delel])` produce un arreglo vacío de tipo flotante. Usarlo como índice en `new_adj[neigs]` provoca un error en NumPy; por tanto, el listado actual no implementa correctamente el caso vacío, aunque el procedimiento sobre conjuntos tenga sentido.

Segundo, `np.delete` mantiene el orden original de los índices conservados en la matriz, mientras que `list(set(...).difference(...))` no garantiza ese mismo orden para las coordenadas. Para obtener una entrada válida para los modelos, ambas salidas deben usar una enumeración común. Esta correspondencia debe verificarse o corregirse en la implementación antes de interpretar el par como una gráfica con atributos. Asimismo, el DFS recursivo está sujeto al límite de recursión de Python. Estas limitaciones no se resuelven mediante una afirmación de que el método siempre alcanza el orden solicitado.

4.7. Análisis de complejidad

Se utilizan $\nu_G = |V|$ y $\varepsilon_G = |E|$ conforme a la Definición 24, y $s = \text{esfera_size}$. El análisis corresponde a matrices densas, dimensión espacial tres y ejecuciones que concluyen, suponiendo aritmética de costo constante sin desbordamiento y el costo esperado habitual de las operaciones con conjuntos de Python. No se consideran constantes ni s ni el número de nodos eliminados.

La construcción del árbol combina el procesamiento de la matriz densa, de costo $\mathcal{O}(\nu_G^2)$, con el ordenamiento de aristas de Kruskal, acotado por $\mathcal{O}(\varepsilon_G \log(\max\{2, \nu_G\}))$ [2]. Convertir el resultado a matriz y simetrizarlo requiere nuevamente $\mathcal{O}(\nu_G^2)$. Aunque DFS con listas de adyacencia tiene costo $\mathcal{O}(\nu_G + \varepsilon_G)$, la implementación busca vecinos mediante `np.where` sobre una fila completa de longitud ν_G por nodo visitado. Su costo es entonces $\mathcal{O}(\nu_G^2)$. La diferencia depende de las representaciones de las Definiciones 28 y 29.

El ciclo principal tiene a lo más ν_G iteraciones porque cada una retira un elemento de la cola sin reintroducirlo. Calcular el centroide y las distancias a éste cuesta $\mathcal{O}(\nu_G)$ por iteración; ordenar la cola cuesta $\mathcal{O}(\nu_G \log(\max\{2, \nu_G\}))$. Generar s puntos cuesta $\mathcal{O}(s)$ en esperanza; cada intento del rechazo se acepta con probabilidad $\pi/4$. No existe una cota determinista para el número de intentos de ese ciclo aleatorio. El arreglo de distancias entre todos los nodos y todos los puntos auxiliares requiere $\mathcal{O}(\nu_G s)$ tiempo y memoria.

La búsqueda en una fila de A cuesta $\mathcal{O}(\nu_G)$. Para la intersección ordenada se obtiene la cota $\mathcal{O}(\nu_G \log(\max\{2, \nu_G\}))$. El filtrado mediante pertenencia a `delel`, que es una lista en Python, puede costar $\mathcal{O}(\nu_G^2)$ por iteración. Las conversiones a conjuntos y la extracción del primer elemento de la lista también tienen costo dependiente de su longitud.

Para cada nodo eliminado, la búsqueda de J_n examina una fila y la suma de sus filas seleccionadas puede procesar hasta ν_G^2 entradas; añadir el vector a una fila y columna cuesta $\mathcal{O}(\nu_G)$. Como cada nodo se marca a lo más una vez, estas transferencias suman $\mathcal{O}(\nu_G^3)$ en toda la ejecución, no por iteración. Poner a cero filas y columnas de los lotes acumula $\mathcal{O}(\nu_G^2)$. Al finalizar, borrar filas y columnas de una matriz densa y binarizarla cuesta $\mathcal{O}(\nu_G^2)$, mientras que extraer las coordenadas cuesta $\mathcal{O}(\nu_G)$.

Una cota superior del tiempo esperado que reúne estas operaciones es

$$\mathcal{O}(\varepsilon_G \log(\max\{2, \nu_G\}) + \nu_G^3 + \nu_G^2 s),$$

con memoria auxiliar $\mathcal{O}(\nu_G^2 + \nu_G s)$. Cuando s se fija independientemente de la entrada, el término cúbico domina esta cota; si s crece, debe mantenerse su contribución. Se trata de una cota superior del código matricial, no de una medición experimental de tiempos.

El número de aristas y la densidad afectan el costo computacional, pero no establecen equivarianza a permutaciones. Esta última propiedad, definida en la Definición 69, requiere estudiar la transformación de la salida al reindexar la entrada; no se deduce de que $\varepsilon_G < \nu_G^2$ ni de las propiedades de las capas neuronales. El recorrido, la selección y la truncación por índices tampoco la garantizan.

La salida pretendida del procedimiento es una gráfica reducida con coordenadas de nodos conservados y adyacencias actualizadas, previa comprobación de su correspondencia de índices. No es por sí misma una superficie triangulada ni una prueba de conservación topológica. Esta representación permite estudiar el efecto del muestreo sobre la entrada de los modelos; su evaluación y las arquitecturas específicas se presentan en el Capítulo 5.

Capítulo 5

Resultados

El problema de clasificación topológica en superficies tridimensionales exige modelos capaces de distinguir entre clases que difieren en invariantes globales como el género o la característica de Euler. En este contexto, EuLearn [5] constituye un referente experimental especialmente apropiado, ya que proporciona superficies con propiedades topológicas controladas y permite evaluar si una arquitectura es capaz de capturar información global más allá de rasgos geométricos locales. Los modelos utilizados en la evaluación —FNO, DGCNN, PointNet, PointNet++, atención y sus variantes con reducción— forman parte del desarrollo experimental reportado en el artículo base, mientras que la contribución del presente escrito consiste en verificar si la representación reducida de la gráfica obtenida en el Capítulo 4 mejora la capacidad de clasificación de estas arquitecturas.

La comparación se realiza en un marco experimental cuya intención no es reemplazar el trabajo original de EuLearn, sino ponerlo en relación con la estrategia de reducción estructurada propuesta en esta tesis. En consecuencia, el objetivo de este capítulo es comprobar si la reducción de nodos y la preservación de relaciones de adyacencia mejoran las métricas de clasificación sin perder información topológica relevante para distinguir superficies con distintos invariantes globales.

5.1. Marco experimental

El conjunto de datos EuLearn ofrece una prueba apropiada para la clasificación de superficies asociadas a distintas propiedades topológicas. En lugar de centrarse en reconocimiento de objetos o en formas geométricas aisladas, la base de datos está diseñada para evaluar si un modelo puede distinguir entre clases que se diferencian por características globales, como el género o la característica de Euler. En este sentido, la tarea experimental no consiste únicamente en aprender una representación geométrica de la superficie, sino en reconocer invariantes que dependen de la estructura combinatoria y de la conectividad del objeto.

La comparación realizada en este capítulo se fundamenta en el marco experimental desarrollado por EuLearn, pero la tesis incorpora una nueva dimensión: la evaluación de la representación reducida generada mediante el método de muestreo estructurado descrito en el capítulo anterior. De esta manera, se comparan dos escenarios: el primero, correspondiente a la entrada clásica del modelo; el segundo, en el que la superficie se representa mediante una gráfica reducida que conserva las relaciones de adyacencia más relevantes. Este diseño permite verificar si la estrategia de reducción contribuye a mejorar la capacidad de clasificación de las arquitecturas utilizadas.

5.2. Arquitecturas utilizadas

A lo largo de la evaluación de las superficies contenidas en la base de datos [5], se utilizan diversas arquitecturas de aprendizaje profundo, cada una con características particulares que las hacen adecuadas para procesar datos tridimensionales. A continuación, se describen brevemente las principales arquitecturas empleadas, así como las variantes que incorporan la representación reducida propuesta en esta tesis.

5.2.1. Fourier Neural Operator

El modelo Fourier Neural Operator (FNO) es una arquitectura diseñada para aprender operadores entre espacios de funciones. En el contexto de EuLearn, esta arquitectura se utiliza sobre la representación continua de la superficie, en lugar de operar directamente sobre la nube de puntos o la malla triangular. Su idea principal consiste en transformar información geométrica en un conjunto de frecuencias mediante la transformada de Fourier, aprender qué patrones de esas frecuencias son relevantes para la clasificación y, posteriormente, transformar nuevamente la información al espacio original mediante la transformada inversa de Fourier. De esta forma, la estrategia permite capturar relaciones globales en la superficie, lo cual resulta especialmente importante para distinguir objetos con distintas propiedades topológicas.

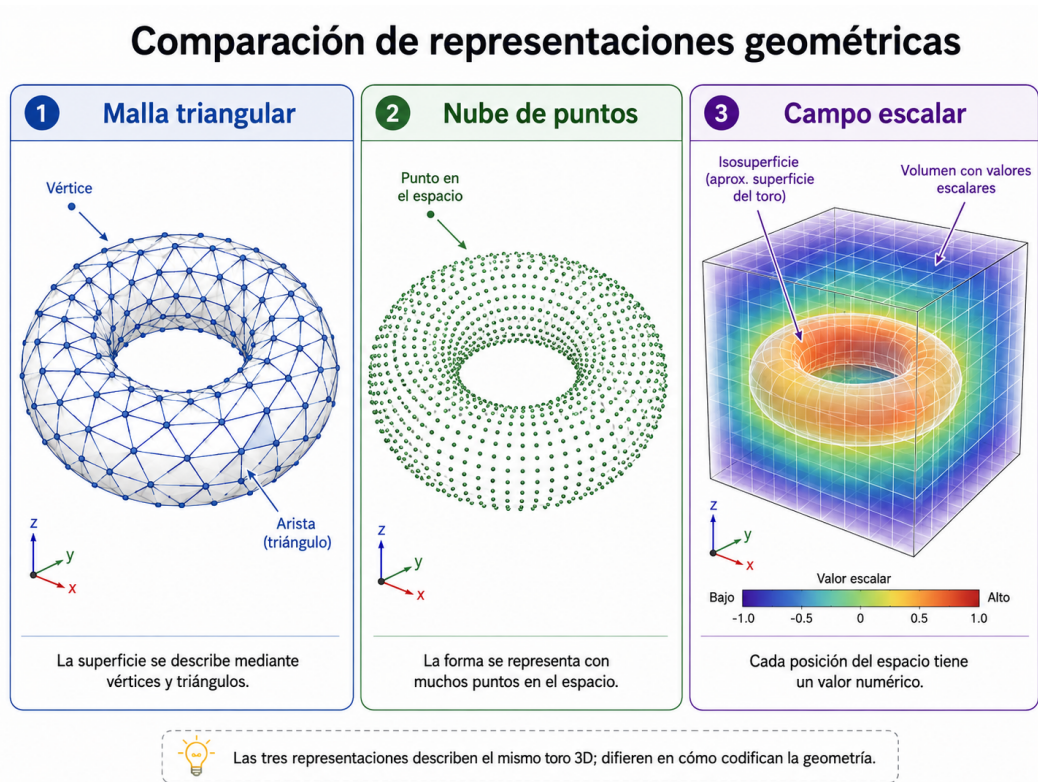


Figura 5.1: Comparación entre malla triangulada, nube de puntos y campo escalar.

De manera general, un bloque FNO puede escribirse como

$$\sigma \left(W v_t(x) + \mathcal{F}^{-1} (R_\theta \cdot \mathcal{F}(v_t)) (x) \right),$$

donde (v_t) es la representación de entrada en la capa (t) , (W) es una transformación lineal local, $(\mathcal{F})$ y $(\mathcal{F}^{-1})$ denotan la transformada de Fourier y su inversa, respectivamente, (R_θ) es un tensor de pesos aprendibles en el dominio frecuencial y (σ) es una función de activación.

5.2.2. Dynamic Graph CNN

La arquitectura Dynamic Graph CNN (DGCNN) está diseñada para procesar nubes de puntos mediante convoluciones definidas sobre gráficas dinámicas. Para cada punto, la red identifica los vecinos más cercanos, usualmente mediante k -nearest neighbors, y los conecta formando una gráfica. A partir de esta estructura, la red analiza la relación entre cada punto y sus vecinos para aprender características locales de la forma de la superficie. Además, estas conexiones pueden recalcularse durante las distintas capas de la red, por lo que la gráfica cambia dinámicamente conforme el modelo aprende nuevas representaciones.

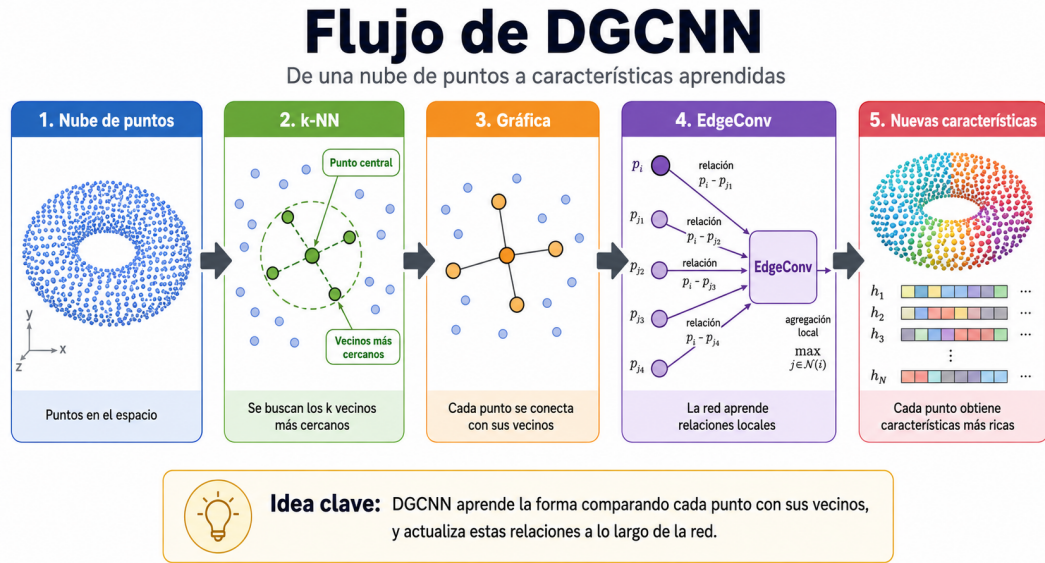


Figura 5.2: Flujo de la operación EdgeConv en DGCNN.

La operación central de DGCNN es EdgeConv, definida como

$$\max_{j \in \mathcal{N}k(i)} \phi \theta \left(h_i^{(l)}, h_j^{(l)} - h_i^{(l)} \right),$$

donde $(h_i^{(l)})$ representa la característica del punto (i) en la capa (l) , $(\mathcal{N}k(i))$ es el conjunto de sus (k) vecinos más cercanos, $(\phi \theta)$ es una función aprendible, usualmente implementada mediante una red MLP, y la operación (máx) agrega la información de los vecinos.

5.2.3. PointNet

PointNet es una arquitectura diseñada para procesar directamente nubes de puntos. Cada punto contiene normalmente coordenadas tridimensionales (x, y, z) , por lo que no es necesario transformar previamente el objeto en imágenes o vóxeles. Su principal propiedad es la invariancia ante permutaciones, es decir, el resultado no depende del orden en que se presenten los puntos de entrada. Esto se logra mediante el uso de operaciones simétricas, como la función $\max$, que agregan información de todos los puntos sin importar el orden en que aparecen. De este modo, PointNet puede obtener una representación global de la forma que posteriormente se utiliza para tareas de clasificación o segmentación.

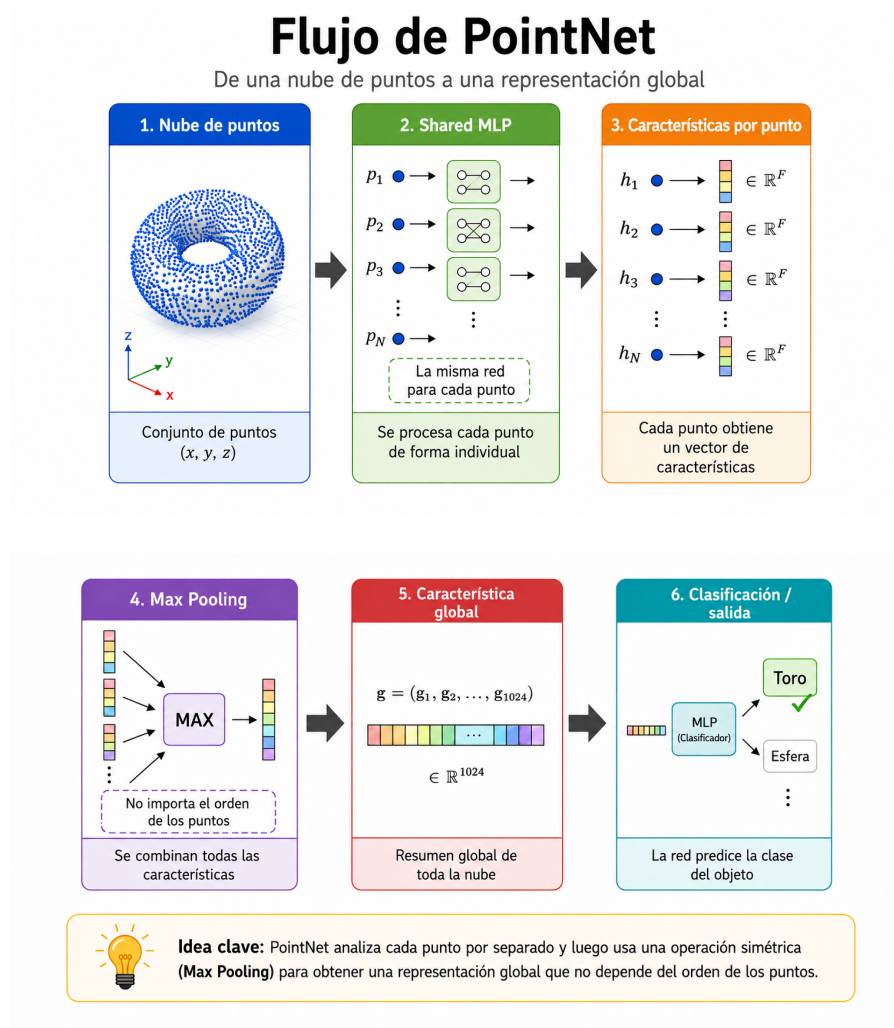


Figura 5.3: Flujo de la arquitectura PointNet. La primera imagen muestra la transformación de cada punto mediante una red MLP, mientras que la segunda ilustra la agregación de características mediante la operación simétrica $\max$ y la posterior clasificación mediante otra red MLP.

Dada una nube de puntos

$$X = x_1, x_2, \dots, x_n, \quad x_i \in \mathbb{R}^3,$$

PointNet aproxima una función sobre conjuntos mediante

$$f(X) \approx \gamma \left(\max_{i=1, \dots, n} \phi(x_i) \right),$$

donde (ϕ) transforma cada punto de manera independiente, $(\max)$ es una operación simétrica que garantiza invariancia al orden, y (γ) es una red que produce la representación global para la clasificación.

5.2.4. PointNet++

PointNet++ extiende PointNet incorporando una estructura jerárquica de agrupamiento local. En lugar de procesar todos los puntos de manera completamente independiente, esta arquitectura selecciona algunos puntos representativos y construye alrededor de ellos vecindades locales. Cada una de estas vecindades representa una pequeña región de la superficie y se procesa mediante una versión local de PointNet para extraer características de esa zona. Después, estas regiones pueden volver a agruparse para formar regiones más grandes, de modo que PointNet++ aprende características a diferentes escalas y captura tanto información local como global.

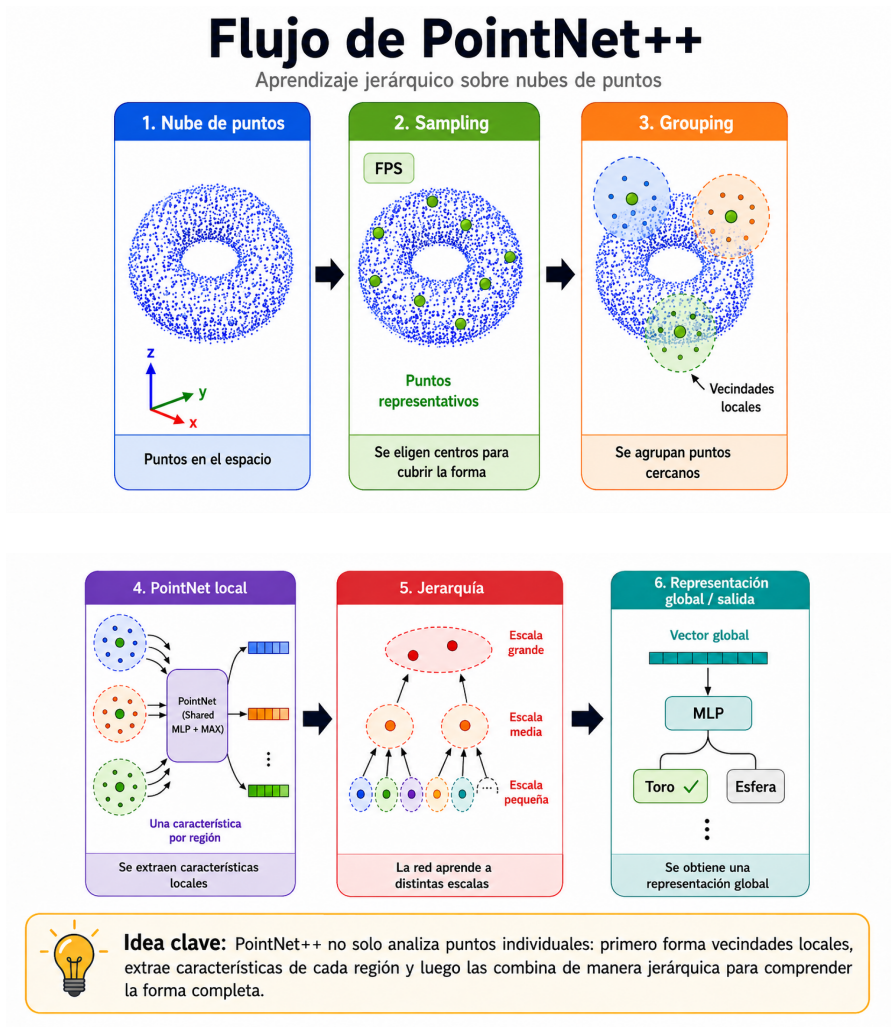


Figura 5.4: Flujo de la arquitectura PointNet++.

De forma esquemática, para un punto central (x_i), se considera una vecindad local

$$x_j \in X : |x_j - x_i| \leq r,$$

o bien una vecindad definida por (k) vecinos más cercanos. La representación local puede expresarse como

$$\gamma \left(\max_{x_j \in \mathcal{N}(x_i)} \phi(x_j - x_i) \right),$$

donde $(x_j - x_i)$ representa la posición relativa de los puntos vecinos respecto al centroide de (x_i) . Posteriormente, estas representaciones locales se combinan jerárquicamente para obtener una representación global de la nube de puntos.

5.2.5. Variantes con reducción estructural

Las variantes GS PointNet y GS Attention corresponden a las adaptaciones de los modelos clásicos que incorporan la representación reducida obtenida mediante el método pro-

puesto en el capítulo anterior. En ambos casos, el objetivo es aprovechar la información de adyacencia y la conectividad local de la gráfica, manteniendo un conjunto de nodos más pequeño pero suficientemente representativo para distinguir superficies con distinto género y característica de Euler. La diferencia principal respecto a los modelos estándar radica en la forma en que se alimenta la arquitectura: en lugar de operar sobre la nube de puntos completa, se utiliza una versión estructuralmente reducida que conserva relaciones relevantes de vecindad y conectividad.

5.3. Análisis de resultados

Con el objetivo de evaluar la efectividad del método de optimización propuesto en esta tesis, se realizaron diversos experimentos sobre el conjunto de datos EuLearn, el cual contiene superficies tridimensionales con distintas propiedades topológicas. La comparación se centró en la capacidad de diferentes arquitecturas para distinguir clases con distintos valores de género y característica de Euler, utilizando las métricas de precisión, exhaustividad, medida F_1 y exactitud.

La Tabla 5.1 presenta los resultados obtenidos. En ella se comparan modelos tradicionales basados en atención, operadores neuronales y procesamiento de nubes de puntos con las versiones que incorporan la reducción estructural propuesta en este trabajo. El propósito de esta comparación no es únicamente observar cuál arquitectura obtiene un mejor rendimiento absoluto, sino verificar si la representación reducida favorece la discriminación de propiedades topológicas globales.

Cuadro 5.1: Comparación de resultados obtenidos con distintos modelos. Las variantes GS corresponden a arquitecturas que incorporan el método de optimización y preservación estructural propuesto en esta tesis.

Modelo	Precisión	Recall	F_1	Accuracy
Attention (clásico)	0.01	0.10	0.02	0.10
FNO	0.02	0.11	0.04	0.20
DGCNN	0.07	0.14	0.07	0.16
PointNet (clásico)	0.50	0.49	0.43	0.49
PointNet++	0.50	0.54	0.52	0.63
GS PointNet (propuesto)	0.82	0.79	0.78	0.79
GS Attention (propuesto)	0.81	0.81	0.79	0.81

Los resultados muestran que las arquitecturas tradicionales presentan dificultades para capturar propiedades topológicas globales de las superficies. Esto se observa de manera especialmente clara en los modelos basados en atención y en DGCNN, cuyos valores de precisión y exactitud permanecen por debajo del 20 %. En cambio, PointNet y PointNet++ muestran una mejora notable, alcanzando exactitudes de 49 % y 63 %, respectivamente. Este comportamiento sugiere que la naturaleza geométrica y la estructura de vecindad de las nubes de puntos contribuyen de manera importante a la clasificación, aunque aún no resultan suficientes para capturar de forma robusta invariantes topológicos globales.

La contribución principal de este trabajo se observa en las variantes GS PointNet y GS Attention. Ambas arquitecturas incorporan la estrategia de reducción estructurada pro-

puesta en esta tesis y obtienen los mejores resultados del experimento. En particular, GS Attention alcanza una exactitud del 81 % y una medida F_1 de 0.79, mientras que GS PointNet obtiene una precisión de 0.82 y una exactitud del 79 %. Estos valores representan mejoras significativas respecto a sus versiones clásicas, así como respecto a PointNet++.

Comparando directamente PointNet++ con GS Attention, se observa un incremento absoluto de 18 puntos porcentuales en exactitud, al pasar de 63 % a 81 %. Este aumento equivale a una mejora relativa aproximada del 28.6 %. Asimismo, la medida F_1 aumenta de 0.52 a 0.79, lo que indica una mejor capacidad para identificar correctamente las distintas clases topológicas presentes en el conjunto de datos. Este comportamiento sugiere que la representación reducida no elimina información discriminativa esencial, sino que favorece la extracción de rasgos más útiles para la clasificación topológica.

En conjunto, estos resultados permiten sostener que el método de optimización propuesto contribuye de manera significativa a la preservación de información estructural relevante para el aprendizaje. La mejora observada en todas las métricas evaluadas indica que la estrategia desarrollada facilita la extracción de características topológicas discriminativas, permitiendo a las redes neuronales capturar propiedades globales de la superficie con mayor efectividad. No obstante, este tipo de evidencia debe interpretarse como un beneficio empírico de la representación reducida, y no como una demostración formal de equivalencia topológica entre la gráfica original y la gráfica reducida.

5.4. Conclusiones

Los resultados obtenidos en este capítulo permiten concluir que la estrategia de reducción propuesta en el Capítulo 4 mejora de manera consistente el desempeño de los modelos evaluados sobre EuLearn. En particular, las variantes GS PointNet y GS Attention superan a los modelos clásicos en precisión, exhaustividad, medida F_1 y exactitud, lo que sugiere que la representación reducida conserva información de adyacencia y vecindad que resulta útil para la clasificación de superficies con distintas propiedades topológicas.

La comparación muestra además que, si bien PointNet y PointNet++ ya representan un avance considerable respecto a arquitecturas menos especializadas, su desempeño sigue siendo inferior al de las variantes que incorporan la reducción estructural. Esto indica que la información de conectividad y la compacidad de la representación juegan un papel importante en la discriminación topológica. En otras palabras, la reducción no parece ser un simple proceso de eliminación de nodos, sino una etapa de selección estructurada que favorece la extracción de rasgos más relevantes para la clasificación.

La principal aportación del capítulo es, por tanto, experimental y metodológica: la evidencia acumulada sugiere que el procedimiento de muestreo y reducción desarrollado en la tesis mejora la capacidad de los modelos para aprender propiedades globales asociadas al género y a la característica de Euler. Sin embargo, la interpretación de estos resultados debe mantenerse rigurosa: la mejora observada se entiende como una mejora empírica en la tarea de clasificación, y no como una demostración formal de conservación topológica en sentido estricto. En ese marco, el trabajo presentado confirma que la representación reducida es una herramienta útil para la clasificación topológica en el contexto establecido por EuLearn y abre una base sólida para futuras evaluaciones más amplias.

Bibliografía

- [1] Michael M. Bronstein, Joan Bruna, Taco Cohen, and Petar Veličković. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. *arXiv preprint arXiv:2104.13478*, 2021.
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, MA, 3rd edition, 2009.
- [3] Carlos Prieto de Castro. *Topología básica*. Ciencia y Tecnología. Fondo de Cultura Económica, México, 2003.
- [4] Reinhard Diestel. *Graph Theory*. Springer, Graduate Texts in Mathematics, 2017.
- [5] Rodrigo Fritz, Pablo Suárez-Serrato, Victor Mijangos, Anayanzi D. Martinez-Hernandez, and Eduardo Ivan Velazquez Richards. Eulearn: A 3d database for learning euler characteristics. *arXiv preprint*, 2025.
- [6] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. <http://www.deeplearningbook.org>.
- [7] William L. Hamilton. *Graph Representation Learning*. McGill University Press, 2020.
- [8] Tero Harju. *Graph Theory*. Department of Mathematics University of Turku, 2007.
- [9] D Knuth. *The Art of Computer Programming*. ADDISON-WESLEY An Imprint of Addison Wesley Longman, Inc., 1998.
- [10] E. G. Minian. *Complejos Simpliciales y Poliedros (Introducción): Notas – Ejercicios Guiados*. Departamento de Matemática, Facultad de Ciencias Exactas y Naturales, Universidad de Buenos Aires.
- [11] James R. Munkres. *Topology*. Prentice Hall, Upper Saddle River, NJ, 2 edition, 2000.
- [12] Andrew Pressley. *Elementary Differential Geometry*. Springer Undergraduate Mathematics Series, 2010.
- [13] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations*, 2019.

Capítulo A

Definiciones básicas

Definición 70. (Espacio métrico) Un espacio métrico consta de un conjunto X y una función $d : X \times X \rightarrow \mathbb{R}^+$, llamada métrica, o distancia que satisface los siguientes axiomas:

1. $d(x, y) = 0$ si y sólo si $x = y$.
2. $d(x, y) = d(y, x)$ para todo $x, y \in X$.
3. $d(x, y) \leq d(x, z) + d(z, y)$ para todo $x, y, z \in X$

Definición 71. (Bola abierta) Sea $x \in X$ con X espacio métrico, $\epsilon \in \mathbb{R}^+$. Definimos la bola abierta con centro en x y radio ϵ como:

$$\mathcal{B}_\epsilon(x) = \{y \in X \mid d(x, y) < \epsilon\}$$

Definición 72. (Vecindad) Sea X un espacio métrico y U un subconjunto de este, decimos que $U \subseteq X$ es una **vecindad** de x si existe $\epsilon > 0$ tal que $\mathcal{B}_\epsilon(x) \subseteq U$

Definición 73. (Conjunto abierto) Sea X un espacio métrico. Decimos que un subconjunto A de X es **abierto** si A es vecindad de x para toda $x \in A$. Es decir $A \subseteq X$ es abierto si y sólo si para todo $x \in A$ existe $\epsilon > 0$ tal que $\mathcal{B}_\epsilon(x) \subseteq A$

Teorema 6. (Unión e intersección de abiertos) Sea $\mathcal{A}$ el conjunto de todos los abiertos en un espacio métrico X . Entonces se cumplen las siguientes afirmaciones:

1. Sea $\mathcal{I}$ un conjunto arbitrario de índices. Si $\{A_i\}_{i \in \mathcal{I}}$ es una familia de elementos en $\mathcal{A}$, entonces $\bigcup_{i \in \mathcal{I}} A_i$ es elemento de $\mathcal{A}$
2. Sea $\mathcal{I}$ un conjunto finito de índices. Si $\{A_i\}_{i \in \mathcal{I}}$ es una familia de elementos en $\mathcal{A}$, entonces $\bigcap_{i \in \mathcal{I}} A_i$ es elemento de $\mathcal{A}$

Capítulo B

Código implementado en Python

Listing B.1: Algoritmo principal de muestreo con preservación de adyacencias

```
1 import numpy as np
2 from copy import deepcopy
3 from scipy.sparse.csgraph import minimum_spanning_tree
4
5
6 def generaPuntosUniformes(h, k, l, num_puntos):
7     """
8     Genera puntos uniformemente distribuidos
9     dentro de una esfera unitaria centrada
10    en (h, k, l).
11    """
12
13    puntos = []
14
15    for _ in range(num_puntos):
16
17        # Metodo de Marsaglia para generar
18        # una direccion uniforme en la esfera
19        while True:
20            V1, V2 = np.random.uniform(-1, 1, 2)
21            S = V1**2 + V2**2
22
23            if 0 < S < 1:
24                break
25
26            a = np.sqrt(1 - S)
27
28            # Direccion sobre la esfera unitaria
29            dx = 2 * V1 * a
30            dy = 2 * V2 * a
31            dz = 2 * S - 1
32
33            # Radio con distribucion proporcional
34            # al volumen de la esfera
35            r = np.random.uniform(0, 1)**(1 / 3)
36
37            # Trasladar el punto al centro (h, k, l)
38            x = h + r * dx
39            y = k + r * dy
```

```

40     z = 1 + r * dz
41
42     puntos.append([x, y, z])
43
44     return np.array(puntos)
45
46
47 def sampling(x, adjacency, size=1000, alcance=1.0, esfera_size=50):
48     """
49     Reduce el numero de nodos de una grafica
50     conservando informacion de adyacencia.
51
52     Args:
53         x:
54             Coordenadas de los nodos.
55
56         adjacency:
57             Matriz de adyacencia original.
58
59         size:
60             Numero objetivo de nodos.
61
62         alcance:
63             Distancia maxima utilizada para
64             identificar vecinos espaciales.
65
66         esfera_size:
67             Numero de puntos utilizados para
68             construir la region de busqueda.
69
70     Returns:
71         new_x:
72             Coordenadas de los nodos conservados.
73
74         new_adj:
75             Matriz de adyacencia reducida.
76     """
77
78     # -----
79     # 1. Construccion del arbol generador minimo (MST)
80     # -----
81
82     mst = minimum_spanning_tree(adjacency).toarray().astype(int)
83
84     # Convertir el MST en una estructura simetrica
85     mst = mst + mst.T
86
87     # -----
88     # 2. Recorrido DFS sobre el MST

```

```

89  # -----
90
91  def dfs_order(node, visited, order):
92      visited[node] = True
93      order.append(node)
94
95      neighbors = np.where(mst[node] == 1)[0]
96
97      for neighbor in neighbors:
98          if not visited[neighbor]:
99              dfs_order(neighbor, visited, order)
100
101  visited = np.zeros(adjacency.shape[0], dtype=bool)
102  order = []
103
104  # Comenzar el recorrido desde el nodo 0
105  dfs_order(0, visited, order)
106
107  # -----
108  # 3. Inicializacion del muestreo
109  # -----
110
111  new_adj = deepcopy(adjacency)
112
113  # Orden inicial inducido por DFS
114  queue = list(order)
115
116  # Nodos que seran eliminados
117  delel = []
118
119  M = adjacency.shape[0]
120
121  # -----
122  # 4. Proceso iterativo de reduccion
123  # -----
124
125  while M - len(set(delel)) > size and len(queue) > 0:
126
127      # Si ya existen nodos eliminados,
128      # reorganizar los candidatos
129      if delel:
130
131          # Nodos que aun permanecen
132          index_selected = list(set(range(M)) - set(delel))
133          selected_coords = x[index_selected]
134
135          # Calcular el centroide de los
136          # puntos que permanecen
137          centroid = selected_coords.mean(axis=0)

```

```

138
139     # Distancia de cada nodo de la
140     # cola respecto al centroide
141     dists = np.linalg.norm(x[queue] - centroid, axis=1)
142
143     # Priorizar los puntos mas alejados
144     queue = [queue[i] for i in np.argsort(dists)[::-1]]
145
146     # -----
147     # 5. Seleccion del nodo central
148     # -----
149
150     idx = queue.pop(0)
151     h, k, l = x[idx]
152
153     # -----
154     # 6. Construccion de la region espacial
155     # -----
156
157     sphere_points = generaPuntosUniformes(h, k, l, esfera_size)
158
159     # Calcular la distancia de cada nodo
160     # a los puntos de la region generada
161     distances = np.linalg.norm(
162         x[:, None] - sphere_points[None, :], axis=2
163     )
164
165     # Nodos que cumplen la condicion
166     # de proximidad espacial
167     sphere_neighbors = np.where(
168         distances.min(axis=1) <= alcance
169     )[0]
170
171     # -----
172     # 7. Interseccion con la adyacencia
173     # -----
174
175     adjacent_neighbors = np.where(adjacency[idx] == 1)[0]
176
177     # Un nodo se considera candidato unicamente
178     # si cumple proximidad espacial y adyacencia
179     neigs = np.intersect1d(sphere_neighbors, adjacent_neighbors)
180
181     # Ignorar nodos eliminados previamente
182     neigs = np.array([n for n in neigs if n not in delel])
183
184     # Evitar eliminar mas nodos de los
185     # necesarios para alcanzar la medida objetivo
186     remaining = M - len(set(delel)) - size

```

```

187
188     if len(neigs) > remaining:
189         neigs = neigs[:remaining]
190
191     # Eliminar los vecinos seleccionados
192     # de la cola de candidatos
193     queue = list(set(queue).difference(neigs))
194
195     # -----
196     # 8. Redistribucion de adyacencias
197     # -----
198
199     for n in neigs:
200
201         # Vecinos del nodo que sera eliminado
202         nears = list(
203             set(np.where(new_adj[n] == 1)[0]).difference(delel)
204         )
205
206         if len(nears) == 0:
207             continue
208
209         # Reunir las conexiones de
210         # los vecinos del nodo n
211         result = new_adj[nears].sum(axis=0)
212
213         # Transferir las conexiones
214         # hacia el nodo central idx
215         new_adj[idx] += result
216         new_adj.T[idx] += result
217
218         # -----
219         # 9. Eliminacion de los nodos seleccionados
220         # -----
221
222         new_adj[neigs] = 0
223         new_adj.T[neigs] = 0
224
225         delel += list(neigs)
226
227         # Eliminar elementos repetidos
228         delel = list(set(delel))
229
230         # -----
231         # 10. Construccion de la grafica reducida
232         # -----
233
234     new_adj = np.delete(new_adj, delel, axis=0)
235     new_adj = np.delete(new_adj, delel, axis=1)

```

```
236
237     # Binarizar la matriz de adyacencia
238     new_adj = 1 * (new_adj > 0)
239
240     # Eliminar posibles auto-conexiones
241     np.fill_diagonal(new_adj, 0)
242
243     # Indices correspondientes
244     # a los nodos conservados
245     index = list(set(range(M)).difference(dele1))
246
247     # Nueva nube de puntos
248     new_x = x[index, :]
249
250     print(
251         '\nOriginal size: {}\nSampling size: {}'.format(
252             x.shape, new_x.shape
253         )
254     )
255
256     return new_x, new_adj
```